Vision-Based Lane Inference for Unstructured and Structured Indian Roads

Capstone Project Report

Harshwardhan Mukund Mohadikar (2023EBCS353)

Shreyas Bhat K (2023EBCS460)

Programme: BSc Computer Science (Online Mode)

BITS Pilani

Academic Year 2025 to 2026

Internal Supervisor: Dr. Ashok Yemineni

DECLARATION

We hereby declare that this capstone project titled "Vision-Based Lane Inference for Unstructured and Structured Indian Roads" is an original work carried out by us and has not been submitted to any other university or institution for the award of any degree.

All measurements reported here were produced by the evaluation harness included with the source code and can be reproduced from it. Where an earlier draft of this work contained a claim we could not reproduce, we either repeated the measurement or removed the claim. Section 6.3 records four hypotheses that we tested and rejected.

Harshwardhan Mukund Mohadikar (2023EBCS353)

Shreyas Bhat K (2023EBCS460)

Date: 28th August 2026 Place: Dubai, UAE & Mangaluru, Karnataka, India

AUTHOR CONTRIBUTIONS

Both authors worked jointly across both phases of the project. In the first phase, Shreyas Bhat K and Harshwardhan Mukund Mohadikar together explored four approaches to lane inference, each implemented and measured before being set aside: a classical computer vision pipeline; that pipeline extended with HSV thresholding and asphalt detection heuristics; a pipeline built on YOLOP dropped in as it came; and a pipeline built on a corrected YOLOP configuration. None of them reached the accuracy the problem requires on Indian road conditions. All four are carried into the ablation in Section 3.3.1 rather than described and discarded, so the measurement that set each one aside is on the same axis as the system that replaced it.

Building on what those attempts established, both authors then worked together on the capstone implementation: the IDD-trained segmentation network and its training, the metric geometry and lane partitioning, the distilled lane marking head, the temporal filter, the confidence calibration, the evaluation harness and baselines, the web application, and this report.

Both authors carried out the experiments reported in Section 6.3 and agreed the interpretation of the results, including the four further hypotheses rejected there.

DATA, SOFTWARE AND ETHICS STATEMENT

Data

All training and evaluation uses the India Driving Dataset, collected and released by IIIT Hyderabad [1] and obtained from the INSAAN distribution site [20]. IDD is distributed through a registration process and its licence is presented at download time. Neither the dataset nor any subset derived from it is redistributed with this project. `tools/fetch_dataset.py` accepts the archive a registered user downloads and verifies it against `tools/idd_lite_manifest.json`, which records the SHA-256 digest of all 6,833 files that every reported result was measured on. That count is images plus label masks plus lane pseudo-labels across both splits, which is why it exceeds the 1,607 frames of IDD-Lite itself. A reader can therefore confirm they hold the same data without the data being passed on by us.

Third party models

YOLOP [2] was used in two ways, both offline. Its drivable area head was evaluated as a baseline, and its lane marking head was used as a teacher to generate the pseudo-labels described in Section 2.4.5. YOLOP is released by its authors under the MIT licence. Its weights are not redistributed here and are fetched from the authors' repository at use time. No endpoint in the deployed application calls it. MobileNetV3 [3] ImageNet weights come from torchvision under the BSD 3-Clause licence.

Intended use

This is a research prototype. It is not validated for vehicle control, driver assistance, or any safety related function, and it should not be used for one. Section 6.4 records where it degrades: it is trained on daylight, normal lens footage, and its accuracy falls on wide angle sensors in low light. Its metric output rests on an assumed camera height and scales linearly with it. The system declines to answer on 39 of 204 validation frames, and that refusal is a designed behaviour rather than a fault; any use of the output should respect both the refusal and the reported confidence.

The imagery is public road scenes from the source dataset. No personal data was collected for this project, and no attempt is made to identify vehicles, number plates or people. The road user detection reports positions and distances of anonymous objects only.

ABSTRACT

Lane detection systems locate painted road markings. On much of the Indian road network there is no paint, yet the road still carries a lane structure that drivers agree on and navigate by. This project infers that structure instead of detecting it. A semantic segmentation network trained on the Indian Driving Dataset produces a drivable surface mask. The vanishing point of the two road boundaries gives the camera pitch, which rectifies the ground plane to a metric bird's eye view. In that view the carriageway is measured in metres and divided by the lane width that IRC:86 prescribes, so the output is a lane count derived from a design standard rather than an arbitrary subdivision of the visible road. A Kalman filter over the lane geometry, with a majority vote on the discrete count, stabilises the result across video.

The system is built in Python with PyTorch, OpenCV and FastAPI, and runs as a web application with a React interface. The segmentation network uses a MobileNetV3-Large backbone with an LR-ASPP head and an FPN decoder, and carries a second output head for lane markings that was trained by distilling YOLOP, since the Indian Driving Dataset carries no lane annotation of its own.

On the 204 frame IDD-Lite validation split the system reaches 0.9403 drivable IoU and 0.8343 boundary F1, against 0.4270 and 0.2048 for the pipeline it replaces, which we show scores below a fixed trapezoid that never looks at the image. Lane inference produces a metric solution on 165 of 204 frames, 80.9 per cent, with a median road edge error of 0.31 metres over 2,288 boundary samples. Temporal filtering reduces carriageway width jitter by 89 per cent across 150 video sequences drawn from three drives, while raising coverage. Rebuilding the confidence score around signals that actually predict error raises its correlation with measured boundary error from 0.187 to 0.272, so the system can flag its own unreliable frames in advance. The complete pipeline runs at 27 frames per second on a laptop GPU.

Keywords: lane inference, semantic segmentation, inverse perspective mapping, Indian Driving Dataset, unstructured roads, knowledge distillation, Kalman filtering.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Table 1. Abbreviations used in this report.

Abbreviation	**Expansion**
API	Application Programming Interface
BEV	Bird's Eye View
BDD100K	Berkeley DeepDrive 100K dataset
CLAHE	Contrast Limited Adaptive Histogram Equalisation
CV	Computer Vision (here, classical non-learned methods)
DFD	Data Flow Diagram
FPN	Feature Pyramid Network
FPS	Frames Per Second
HSV	Hue, Saturation, Value colour space
IDD	Indian Driving Dataset
IDD-AW	Indian Driving Dataset, Adverse Weather subset
IoU	Intersection over Union
IPM	Inverse Perspective Mapping
IRC	Indian Roads Congress
LR-ASPP	Lite Reduced Atrous Spatial Pyramid Pooling
mIoU	mean Intersection over Union across classes
MPS	Metal Performance Shaders (Apple GPU backend)
SPA	Single Page Application
VP	Vanishing Point
YOLOP	You Only Look Once for Panoptic driving perception

CHAPTER 1: INTRODUCTION

1.1 Overview of the project

A lane detection system finds painted markings and reports where they are. That works on a motorway and fails on a road with no paint. Much of the Indian network has no paint, or has paint worn to the point of being unusable, yet traffic on those roads still organises itself into lanes that drivers recognise. The information is present in the geometry of the road surface rather than in its markings.

This project reads that geometry. A network trained on the Indian Driving Dataset labels every pixel of a forward facing camera frame into seven classes, one of which is the drivable surface. The two boundaries of that surface converge at a vanishing point, and the row on which they converge fixes the camera pitch. With pitch known, the road plane can be rectified to a top down view in which distances are metres rather than pixels. The carriageway is measured there and divided by the lane width that IRC:86 prescribes for the road class, giving a lane count.

The project is delivered as a running web application rather than a set of scripts. It exposes the pipeline through an HTTP API and a React interface with two routes: a technical report and an interactive demonstration that accepts a photograph, a video clip or a live camera feed.

1.2 Problem statement and motivation

Existing lane detection assumes lane markings exist. On unstructured roads that assumption fails, and the failure is silent: a detector returns nothing, or returns noise, and offers no way to tell which. Any downstream use, such as driver assistance, road survey or traffic analysis, then has no lane information at all for the roads where it would be most useful.

The problem this project addresses is therefore narrower and better posed than lane detection: given a single forward facing frame of an Indian road, estimate how many lanes the carriageway supports and where their boundaries lie, whether or not the road is painted, and report an honest confidence alongside the estimate.

Two things make this tractable. First, the Indian Roads Congress publishes design lane widths, so a measured carriageway width implies a lane count without needing to observe the lanes themselves. Second, the metric width can be recovered from a single camera without

calibration data, because, as Section 2.4.2 shows, the lateral scale of the rectified ground plane does not depend on focal length.

1.3 Related work, and why it does not transfer

Lane detection is a well developed field and almost all of it assumes the thing this project cannot assume. Segmentation based methods such as SCNN [15] and LaneNet [16] learn to label marking pixels and then cluster them into instances. Row anchor methods such as Ultra Fast Lane Detection [17] reformulate the task as selecting, for each image row, which horizontal cell contains a marking, which makes them fast but presupposes that some cell does. Parametric methods such as PolyLaneNet [18] regress polynomial coefficients per lane, and CLRNet [19] refines line proposals across feature levels. All of them are trained and evaluated on TuSimple, CULane or BDD100K, which are dominated by marked highways and marked urban roads.

On an unmarked road every one of these degrades in the same way, and it is not a graceful one. The output is empty or arbitrary, and nothing in the formulation distinguishes the two, so a downstream consumer cannot tell absence of markings from failure to find them. Section 3.3.1 measures a version of exactly this: YOLOP, correctly wired, reaches 0.7243 drivable IoU on this data, and its lane marking head returns lane pixels on 67 per cent of frames, which is a statement about how many Indian road frames have visible paint as much as about the model.

Work specifically on unstructured roads is thinner. The IDD authors [1] make the case that Indian road scenes break assumptions built into Cityscapes style datasets, which is the premise this project starts from, but IDD annotates drivable surface rather than lanes, so it supports segmentation rather than lane detection. That absence is not an oversight in the dataset: on a road with no markings there is no ground truth lane to annotate, which is why this project infers structure from geometry and a design standard instead of learning it from labels. The consequence, stated plainly in Section 3.3.3, is that the carriageway extent can be validated against ground truth and the interior divisions cannot.

1.4 Objectives of the capstone

- Replace the colour threshold road detector of the study project with a semantic segmentation network trained on Indian road imagery.

- Recover metric ground plane geometry from a single frame, with the camera pitch measured per frame rather than assumed.
- Infer a lane count from the measured carriageway width and the IRC design width, rather than subdividing the visible road by a fixed rule.
- Detect painted markings where they exist, without a separate model at inference time.
- Stabilise the estimate across video, and show that the stabilisation does not come at the cost of coverage.
- Report a confidence that correlates with measured error, so unreliable frames can be identified in advance.
- Measure everything against baselines, including trivial ones, and report the experiments that failed.
- Deliver the result as a working application that a reader can run.

1.5 Scope of implementation

The system operates on monocular forward facing imagery. It reports the number of lanes on the carriageway ahead, the metric width of the carriageway and of each lane, which lane the camera occupies, the positions of detected road users with distances, and a confidence value. It runs on single images, on uploaded video and on a live camera stream.

Outside the scope: the system does not plan a path, does not control a vehicle, and does not use any sensor other than the camera. It does not attempt lane detection at night or in heavy rain, and Section 6.3 records what we measured when we tried to extend it to a consumer dashcam at dusk. Problem identification and the initial system design were completed in the study project; this report covers the implementation, and states where the earlier design was changed and why.

1.6 Organization of the report

Chapter 2 describes the implementation: the architecture, the technology stack, the modules and the algorithms, with the geometry derived in full. Chapter 3 covers testing and results, including the ablation that compares every configuration against trivial baselines, the accuracy of the inferred geometry against ground truth, and the failure analysis. Chapter 4 covers execution and deployment. Chapter 5 records project execution evidence. Chapter 6 concludes, states the limitations honestly, and lists the work that remains.

CHAPTER 2: IMPLEMENTATION DETAILS

2.1 System architecture and design

The system has four tiers: a browser client, an HTTP application server, the inference modules, and read only storage for weights, sample data and measured results. Figure 1 shows how they fit together. Every request enters through one API surface, and the inference modules hold no state of their own except the per client filter state kept by the session store, which exists because temporal smoothing needs the previous frame.

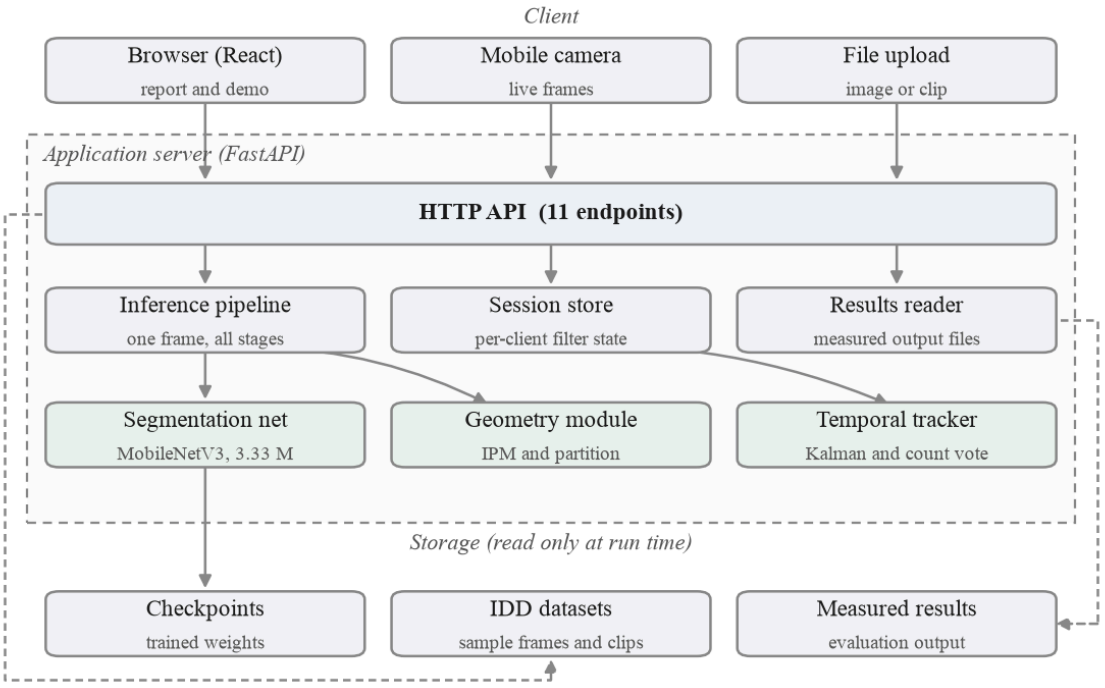

Figure 1. High level system architecture.

Keeping filter state on the server, keyed by a session identifier the client generates, is a deliberate choice with a consequence worth stating: the application needs a process that lives between requests, so it cannot be deployed as stateless serverless functions without moving that state to an external store.

2.1.1 Data flow

Figure 2 traces a single frame through the pipeline. The forward pass produces semantic classes; the drivable class alone feeds the geometry stage. Measurement and stabilisation then run on the rectified plane, and the result returns both as JSON and as a rendered overlay.

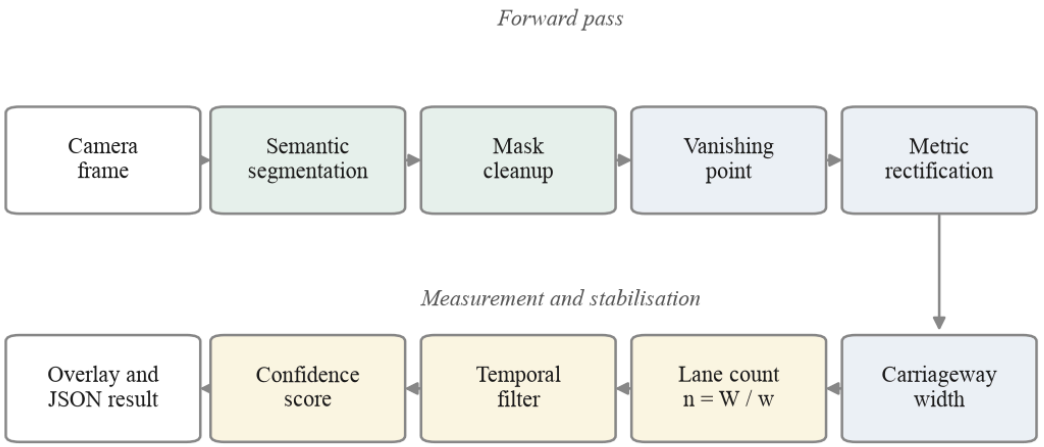

Figure 2. Data flow through the inference pipeline.

2.1.2 Component interaction

Figure 3 shows the exchange for one live camera frame, which is the most involved path because it touches the session store twice. The client holds at most one request in flight and drops frames that arrive while inference is running, rather than queueing them, since a queued frame is stale by the time it is processed.

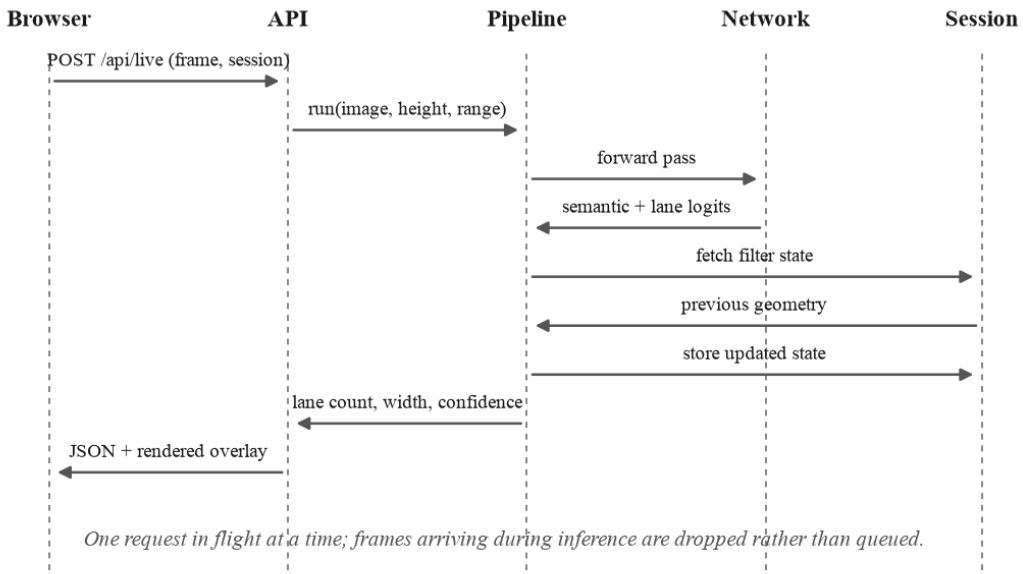

Figure 3. Component interaction for one live camera frame.

2.2 Technology stack

Table 2. Technology stack.

Layer	Choice	Reason

Language	Python 3.12.4	The model, the geometry and the server share one language.
Deep learning	PyTorch 2.13, torchvision 0.28	MobileNetV3 weights ship with torchvision; MPS gives GPU training on a laptop.
Computer vision	OpenCV 5.0	Rectification, morphology, contour and profile work.
Numerics	NumPy 2.5	Polynomial fitting and the Kalman update.
Web server	FastAPI 0.141, Uvicorn 0.52	Typed request parsing and multipart upload with little code.
Frontend	React 19, Vite 8, Tailwind 3	Two routes from one build; the report and the demo share components.
Charts	Recharts 3	Renders the measured results the API serves.
Testing	pytest	55 tests over geometry, metrics, trajectory, API payloads, API robustness and overlay reach.
Training	Apple MPS on the laptop	The shipped weights come from a local run of 127 minutes. An earlier attempt on a Colab T4 was lost; see Section 3.3.2.
Teacher model	YOLOP via torch.hub	Used offline only, to generate lane markings pseudo-labels.

The deployment image excludes the teacher model and its dependencies. No endpoint calls YOLOP at run time, so the container avoids a download of roughly 200 megabytes for a component used only during dataset preparation and evaluation.

2.3 System modules

Table 3. System modules and their responsibilities.

Module	**File**	**Responsibility**
Segmentation backend	core/dl_drivable.py	Loads the checkpoint, normalises input, returns semantic and lane logits.
Classical detector	core/asphalt_detector.py	Adaptive CIELAB and texture road estimate, kept as a gated fallback.
Semantics	core/semantics.py	Class maps, scene plausibility, road user extraction and distances.
Geometry	core/geometry.py	Vanishing point, camera pitch, inverse perspective mapping, calibration.
Lane tracker	core/lane_tracker.py	Road profile, carriageway width, lane partition, Kalman filter, confidence.

Trajectory lanes	core/lane_trajectory.py	Empirical lanes from observed traffic. Implemented, measured, off by default.
Pipeline	core/pipeline.py	Orders the stages and records timings.
Visualiser	core/visualizer.py	Renders every intermediate stage.
Dataset	data/idd.py	IDD loading, label mapping and the augmentation suite.
Model	models/segnet.py	MobileNetV3 backbone, LR-ASPP, FPN decoder, two heads.
Training	models/train.py	Schedule, losses, checkpointing and resume.
Evaluation	eval/	Ablation, geometry accuracy, temporal, sensitivity, failures.
Server	server/api.py	Eleven HTTP endpoints and the static frontend.

2.3.1 Functional flow

Figure 4 shows the order in which the stages run, including the path taken when the scene check declines to answer. Each stage writes an intermediate image, which is what the stage walkthrough in the demonstration displays.

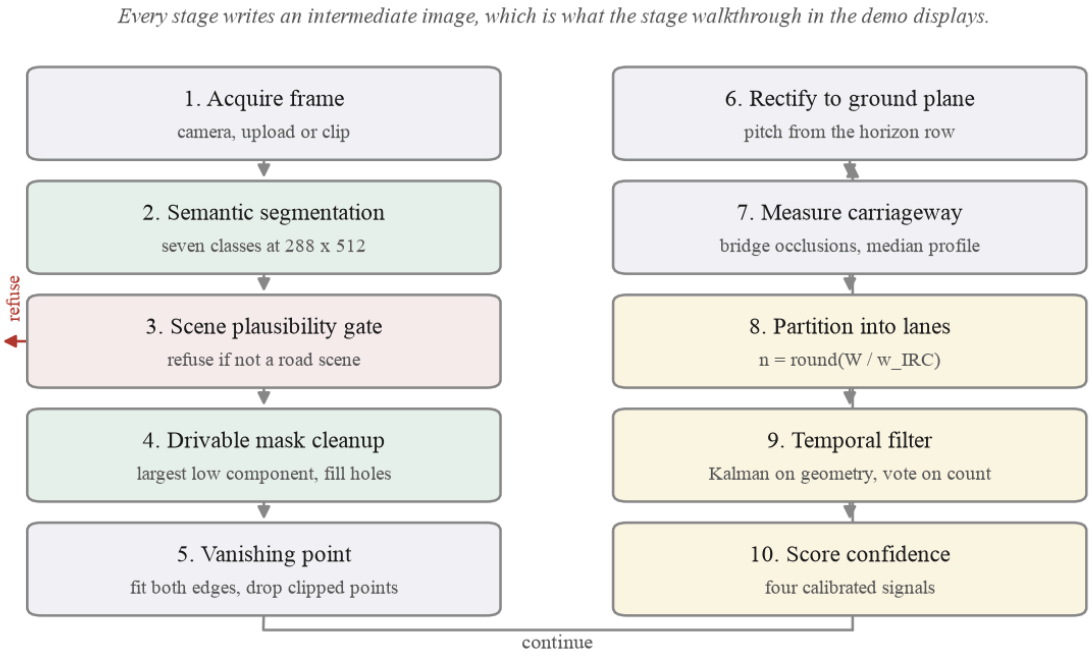

Figure 4. End to end process flow, including the refusal path.

2.4 Key algorithms

2.4.1 Semantic segmentation

The network is a MobileNetV3-Large backbone with a Lite Reduced ASPP context head and a Feature Pyramid Network decoder, 3.33 million parameters in total. Two output heads share the decoder: a seven class semantic head and a binary lane marking head. An auxiliary head on the stride 16 features supplies deep supervision during training and is discarded at inference.

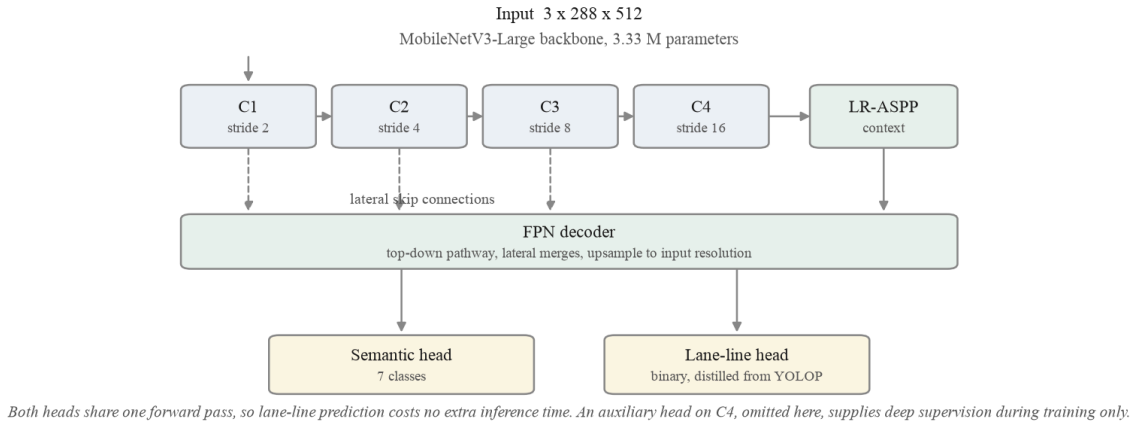

Figure 5. Segmentation network architecture.

A colour threshold cannot separate asphalt from shadow, dust or wet ground, because those differ in the same channels the threshold uses. A network trained on Indian road imagery can, and Section 3.3 measures how much of the overall gain comes from this one change.

2.4.2 Recovering metric scale

This step needs care, because lane inference consumes a metric road width and the dataset ships no camera metadata. Its images were resampled from more than one source resolution, so focal length cannot be read from the files either. The result of this section is that it does not need to be.

Inverse perspective mapping [5] takes a pinhole camera at height H above the road, pitched down by an angle theta, with focal length f in pixels and principal point (cx, cy). A point on the ground at forward distance Y and lateral offset X projects to image row v and column u. For the row, writing the projection and keeping terms to first order in theta, which is justified because Section 2.4.3 bounds the recovered pitch to plus or minus 15 degrees:

$$v - cy = f H / Y - f \tan(theta) \qquad (1)$$

where v is the image row of a ground point at forward distance Y.

The horizon is the limit of that expression as Y grows without bound, so the horizon row v_h satisfies:

$$v\_h - cy = -f \tan(\theta) \qquad (2)$$

so f tan(theta) = cy - v_h: the horizon row measures that product directly.

Substituting (2) into (1) removes the pitch term and leaves forward distance as a function of the row and the horizon row alone:

$$Y = f H / (v - v\_h) \qquad (3)$$

forward distance still depends on f.

For the lateral direction, a pixel width du observed at row v subtends a ground extent dX = du Y / f, again to first order in theta. Substituting (3):

$$dX = du\,(f H / (v - v\_h)) / f = du\, H / (v - v\_h) \qquad (4)$$

the focal length cancels.

The first order step is worth bounding rather than waving at. The terms dropped between the exact projection and (1) amount to replacing Y cos(theta) + H sin(theta) with Y. Within the plus or minus 15 degree pitch bound, and over the 6 to 22 metre range the width is measured on, that substitution changes the lateral scale by less than 1.6 per cent, which is below the 11.8 per cent median width error the system actually exhibits and far below the effect of the camera height prior.

That cancellation is the useful result. It happens because the horizon row already measures f tan(theta) jointly, by (2), and that same product is what sets the ground plane scale. Forward distance, by contrast, keeps its dependence on f in (3), which is why the two behave so differently under a wrong assumption.

Figure 15 in Section 3.3.5 verifies this numerically by rebuilding the transform across a hundred degree sweep of assumed field of view. The recovered lateral span moves by one per cent while the forward distance moves by a factor of nine, exactly as (3) and (4) predict. Because lane inference is a purely lateral operation, it is insensitive to the field of view, and the pipeline is left with exactly one metric free parameter, the camera height.

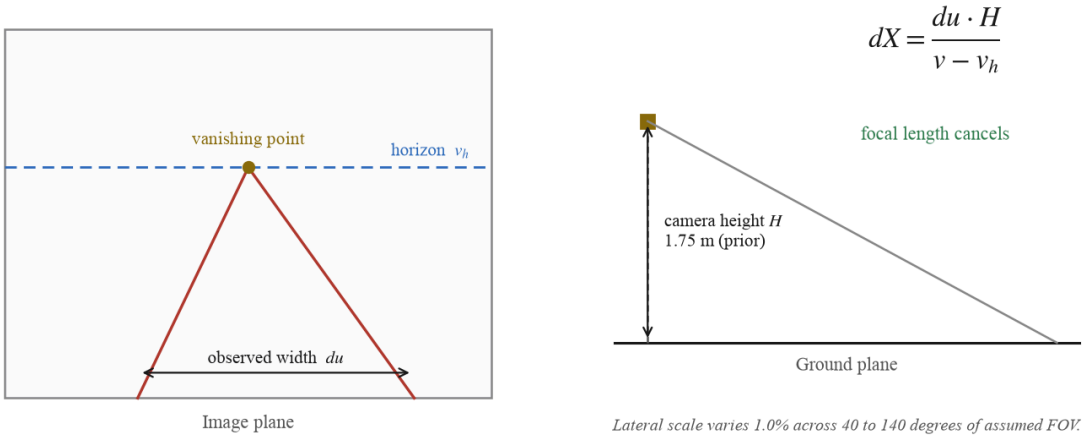

Figure 6. Recovering metric scale from the vanishing point.

Camera height is adopted as a prior of 1.75 metres rather than claimed as a measurement, and an earlier draft of this report defended that prior with an argument we have since had to withdraw. That argument observed a median rectified carriageway of 7.03 metres against the IRC:86 two lane design width of 7.0 metres and read the agreement as support for the height. The 7.03 metre figure was produced by bridging vehicle occlusions across every rectified row, and Section 2.4.4 shows that this inflates the measured carriageway. With the row-wise merge the same split gives a median of 5.23 metres. The agreement was an artefact of the inflation, and it was also the wrong standard to reach for: only 22 per cent of these validation frames carry a carriageway as wide as a 7.0 metre two lane section, so most of this split was never the road that standard describes.

What can be said now is narrower and better evidenced. At the assumed height the median inferred carriageway is 5.23 metres against 5.12 metres measured from the drivable ground truth on the same frames, a difference of 0.11 metres. That is a strong statement about the geometry and it is not a statement about the height, because the ground truth width is itself recovered through the same rectification and scales linearly with the assumed height exactly as the inferred width does. Both would move together under a different prior. The absolute scale therefore still rests on the assumption, and this report does not claim otherwise. The median rectified ground contact width of well resolved vehicles is 1.252 metres from sixteen instances, but IDD merges motorcycles, autorickshaws, cars and buses into one vehicle class, so that anchor cannot be turned into a height without knowing which of them the median instance is. Section 3.3.5 reports a five point sensitivity sweep over the full validation split so a reader can see which conclusions survive a different assumption, and the code provides

`calibrate_from_known_width()` to remove the parameter outright wherever a single true width is known.

2.4.3 Vanishing point and rectification

Both road boundaries are extracted from the drivable mask and fitted, and their intersection gives the vanishing point. One detail matters more than it sounds. Edge points clipped by the frame border must be excluded, because where the carriageway runs off the side of the image the observed boundary is the image border rather than the road, and including those points drives the fit vertical. On validation frame 101 this produced a pitch estimate of 25.1 degrees and a rectified view containing nothing but sky; excluding clipped points corrects it to minus 0.9 degrees. Pitch is also bounded to plus or minus 15 degrees.

2.4.4 Measuring the carriageway and partitioning it

Width is measured on the rectified plane by taking the longest contiguous run of road pixels in each row and reducing the profile to a median. That rule has a failure mode which the measurements exposed: a pair of parked vans reduces the measured carriageway to the width of the gap between them. Validation frame 0 reported a 2.75 metre carriageway that was exactly that gap. Bridging vehicle occlusions before measuring corrects it and raises coverage from 151 to 171 of 204 frames.

Bridging every row is not the right answer either, and measuring it was what exposed the largest single defect in this system. An occluder that spans a central median joins the ego carriageway to the opposing one, the measured road doubles, and the frame comes back with a confident wrong answer rather than a refusal. Across the validation split, bridging every row takes the relative width error at the ninetieth percentile from 34.1 per cent to 151.2. The segmentation was never the problem: measured against ground truth the predicted road mask is within 0.20 metres of the true width at the median, while the lane model built from it was 0.58 metres wider than that same mask.

Neither mask is right for a whole frame, so the shipped pipeline chooses per rectified row. A row the unbridged mask can measure is kept. A row it cannot measure is taken from the bridged mask, which is what recovers coverage. A row both can measure uses the bridged value only when bridging widened it by less than one lane width, since a larger jump means the gap that was closed was wider than any vehicle could account for. The three are set side by side below. It is a trade and not a free win: the unbridged mask alone remains the most

accurate option, and the merge buys 14 of the 20 frames that bridging adds for about one percentage point of median relative error.

Table 4. Relative carriageway width error against ground truth for the three ways of handling vehicle occlusion in the width profile.

Width profile measured on	Solved	Median	Mean	p90
Unbridged mask only	151 / 204	11.0%	16.5%	34.1%
Bridged mask only	171 / 204	18.5%	51.5%	151.2%
Row-wise merge (shipped)	165 / 204	11.8%	22.4%	46.5%

The same bridging applied to the vanishing point stage makes results worse, and Section 6.3.2 reports that result. The two stages therefore use different masks, because they ask different questions: the vanishing point stage asks where the road converges and is best served by the clean unoccluded fragment, while the width stage must see through the traffic standing on the road.

The partition itself is an integer division:

$$n = \mathrm{round}(W / w) \tag{5}$$

W is the measured carriageway width in metres, w the design lane width, and n the reported lane count.

Two details of that division matter and were not obvious. The first is how w is chosen. The pipeline does not classify the scene as urban or rural; it applies a threshold to the width it has already measured:

```
w = 3.50 m   if W >= 6.0 m      # IRC:86 urban lane width
w = 3.00 m   otherwise          # narrow-road value, see below
```

The 3.5 metre figure is the urban lane width in IRC:86 [7]. The 3.0 metre figure is this project's adopted value for narrow carriageways rather than a clause quoted from a standard: rural and village road guidance in India spans a range of carriageway widths, and 3.0 metres was chosen as the narrow-road divisor because it is the smallest width that still exceeds the 2.5 metre plausibility floor. It is a configurable parameter, not a constant of nature, and Section 6.4 records that it should be confirmed against the applicable standard before any deployment claim rests on it.

A carriageway of six metres or more is treated as an urban road and divided by the urban width; anything narrower is treated as a narrow road. The threshold sits at 6.0 m because that is where an urban two lane carriageway (7.0 m nominal) and a rural single lane road stop overlapping. The rule is crude, and it is worth stating plainly that it is a rule about width rather than a judgement about the road: a narrow urban lane and a wide rural one are both classified by size alone.

The second detail is that this width classification is separate from the `road_type` the system also reports. That field records whether painted markings were found, structured against unstructured, and it selects between two different partition methods rather than between two widths. A frame can be unstructured, no markings, and still be partitioned on the urban 3.5 metre basis. The interface shows both, because showing only one of them is misleading.

Finally, a floor: if dividing by w would give lanes narrower than the minimum plausible lane width of 2.5 metres, the count is reduced until they are not. Where even a single lane would fall below that floor the frame is refused rather than answered, which is the largest single refusal cause in Table 14.

The rule is discontinuous at the threshold, which is easiest to see worked through. Note the third row: a carriageway just above the threshold is divided by the larger width, so it yields fewer lanes than one just below it.

Table 5. The width threshold worked through, including the discontinuity at 6.0 m.

Measured W	Divisor w	n = round(W / w)	Resulting lane width
5.5 m	3.00 m	2	2.75 m
5.9 m	3.00 m	2	2.95 m
6.1 m	3.50 m	2	3.05 m
10.5 m	3.50 m	3	3.50 m

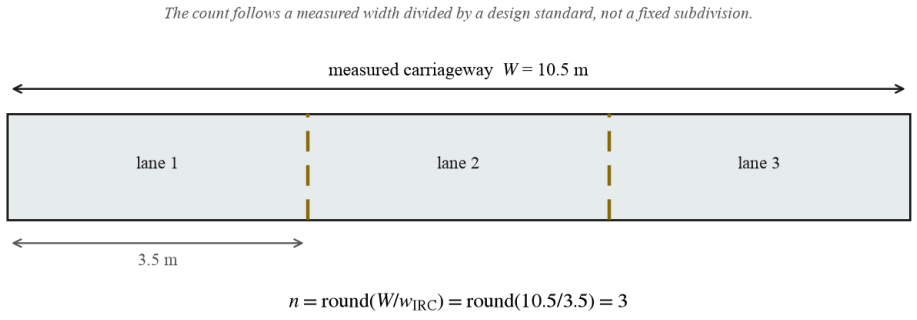

Figure 7. Partitioning a measured carriageway into lanes.

2.4.5 A lane marking head distilled from YOLOP

IDD carries no lane marking annotation, which is the reason the inference problem exists at all. YOLOP ships a trained lane marking head, which the study project never read because it used only the drivable area head. Measured on our validation split, that head returns lane pixels on 67 per cent of frames against 9.4 per cent for the hand written top hat filter we first used.

YOLOP costs roughly 150 milliseconds per frame on CPU against 75 for our network on the same two threads, and it is trained on BDD100K rather than Indian roads, so it is not suitable at inference time. It was instead used as a teacher [11]: 1,607 pseudo-labels were generated over IDD-Lite, and a second head was trained against them jointly with the semantic task. Marked and unmarked roads are now handled in one forward pass at no extra cost.

The split matters here and is worth stating precisely, because 1,607 is the size of IDD-Lite entire. The teacher labelled every frame, but the 1,403 training frames and the 204 validation frames were kept apart exactly as they are for the semantic task: only the training pseudo-labels were used to train the head, and the 204 validation pseudo-labels serve solely as evaluation targets. No validation frame contributed a gradient.

What the resulting lane-marking IoU measures also needs saying. IDD carries no lane annotation, which is the premise of this whole project, so there is no ground truth to score against. The figure in Table 7 is agreement with the teacher on frames the student never trained on. It answers whether distillation transferred, and it is not an accuracy against reality. Figure 8 is the closest thing to direct evidence available.

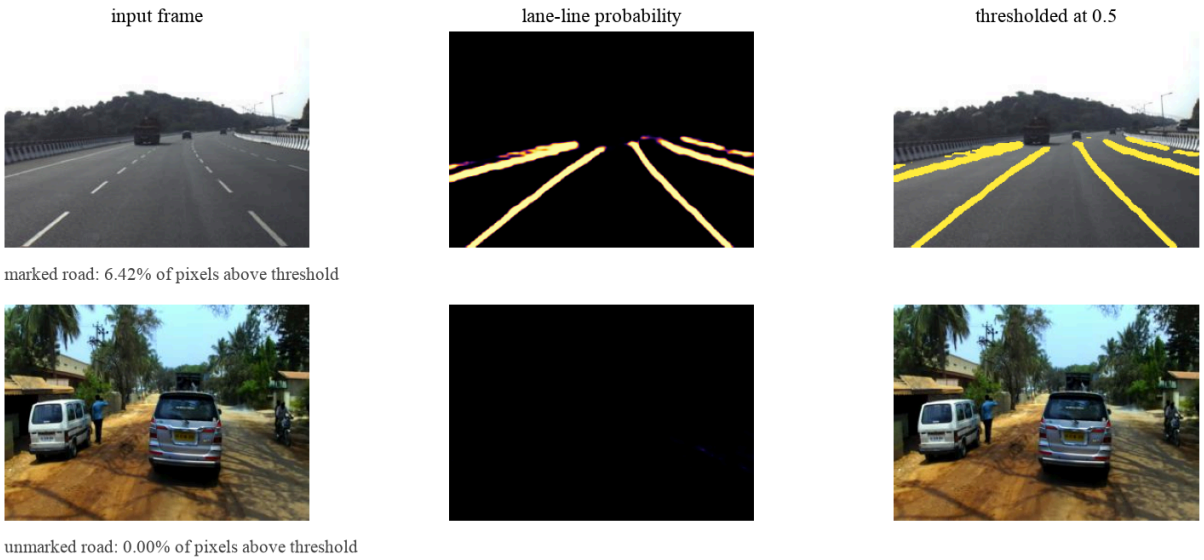

Figure 8. The distilled lane marking head on a marked and an unmarked road.

Figure 8 is the evidence that the head learned the task rather than the teacher's noise. On a marked highway it fires on all four visible lines and on almost nothing else. On an unmarked road it returns 0.00 per cent of pixels above threshold, which is the correct answer and the one that matters: a marking detector that fired on unmarked roads would corrupt the partition rather than improve it, because Section 2.4.4 switches partition method when markings are found.

One observation that is not acted on. The learned head and the hand written top hat filter it replaced are complementary: on the IDD-Lite split the head finds markings on 16 frames the filter misses, and the filter on 23 the head misses. That suggests a combined detector would beat either. The shipped pipeline does not do this, because the objective in Section 1.4 was one model at inference time and the filter is not a live stage; Figure 4 shows no such step. It is recorded as an observation for future work rather than a design decision.

Three timing figures appear in this report and they measure different things, so they are stated together here. The segmentation forward pass alone takes 15.8 milliseconds on the laptop GPU and 75 milliseconds on two CPU threads. The full pipeline, which adds rectification, width measurement, partitioning and the analytics, takes 36.5 milliseconds per frame on the GPU, which is 27 frames per second; Table 5 reports the segmentation figure because that table compares detectors rather than pipelines. The larger number visible in the interface screenshots includes JPEG decoding, rendering five overlay images and transferring them over HTTP, none of which is inference.

2.4.6 Temporal filtering

A Kalman filter [9] runs over the centreline polynomial coefficients and the carriageway width, with per state process and measurement noise. The discrete lane count is not filtered, because averaging a count is meaningless; a majority vote over a short window is used instead. Section 3.3.4 measures the effect over 150 real video sequences.

2.4.7 Confidence, and refusing to answer

Two mechanisms guard the output. The first is a scene check, because a network trained only on roads cannot abstain on its own: ours labels a blank black frame 26 per cent drivable. Before trusting the mask the pipeline tests how many semantic classes are present, since a road scene shows five to seven, and how much drivable surface sits above the horizon, which

should be none. The second is a confidence score rebuilt from signals that actually correlate with error, described with its measurements in Section 3.3.3.

The lane inference step, in outline:

```
        # min_width = 2.50 m; max_width = 8 * 3.50 m + 4.0 m = 32.0 m,
        # where 8 is the largest lane count the partition will consider
        def infer_lanes(profile, ipm, geom):
            W = median_width(profile)      # m, occlusions bridged
            if not (min_width <= W <= max_width):
                  return None                              # refuse rather than
    guess

            xl, xr = fit_edges(profile)            # X = f(Y), metres
            W_ref  = xr(Y_ref) - xl(Y_ref)
            if not (min_width <= W_ref <= max_width):
                centre = 0.5 * (xl(Y_ref) + xr(Y_ref))
                xl, xr = centre - W/2, centre + W/2   # use the profile

            # width threshold, not a scene classification; see 2.4.4
            w = 3.50 if W_ref >= 6.0 else 3.00
            n = max(1, round(W_ref / w))
            while n > 1 and (W_ref / n) < geom.min_lane_width_m:
                n -= 1                                 # 2.5 m floor
            return LaneModel(n_lanes=n, carriageway_width_m=W, ...)
```

The bounds check on `W_ref` was added after a dashcam frame produced a zero width carriageway. Only the profile median had been checked, so where the two fitted edges cross, the second estimate collapsed to its clamp of 1e-6 and propagated through every downstream stage. On that frame the corrected carriageway is 9.03 metres and three lanes, which matches the road.

A third guard applies to the drawing rather than to the answer, and it was added last. The two edge polynomials will evaluate at any distance asked of them, so the overlay was drawn to a fixed reach of 22 metres whether or not a road boundary had been observed that far ahead. Measured across the validation split, that put lane boundaries more than two metres past the last supporting pixel on 14 per cent of frames and more than five metres past on 8 per cent, reaching fourteen metres past the evidence on the worst frame. The lane model now records the furthest distance at which a boundary was genuinely observed, and both the image overlay and the rectified panel stop there. It changes no measured quantity anywhere in this report, which is exactly the point: the picture had been asserting something the measurements never did, and a reader looking at the overlay would have had no way to tell.

2.5 Output and code

Figure 9 shows every stage of the pipeline on four validation frames, chosen to span the range of outcomes rather than at random. The first is a narrow single lane road, the second a two lane road with paint, the third a painted three lane road, and the fourth a frame the system declines to answer because dense traffic hides both road edges. Declining is the correct behaviour there: a wrong lane model is worse than none.

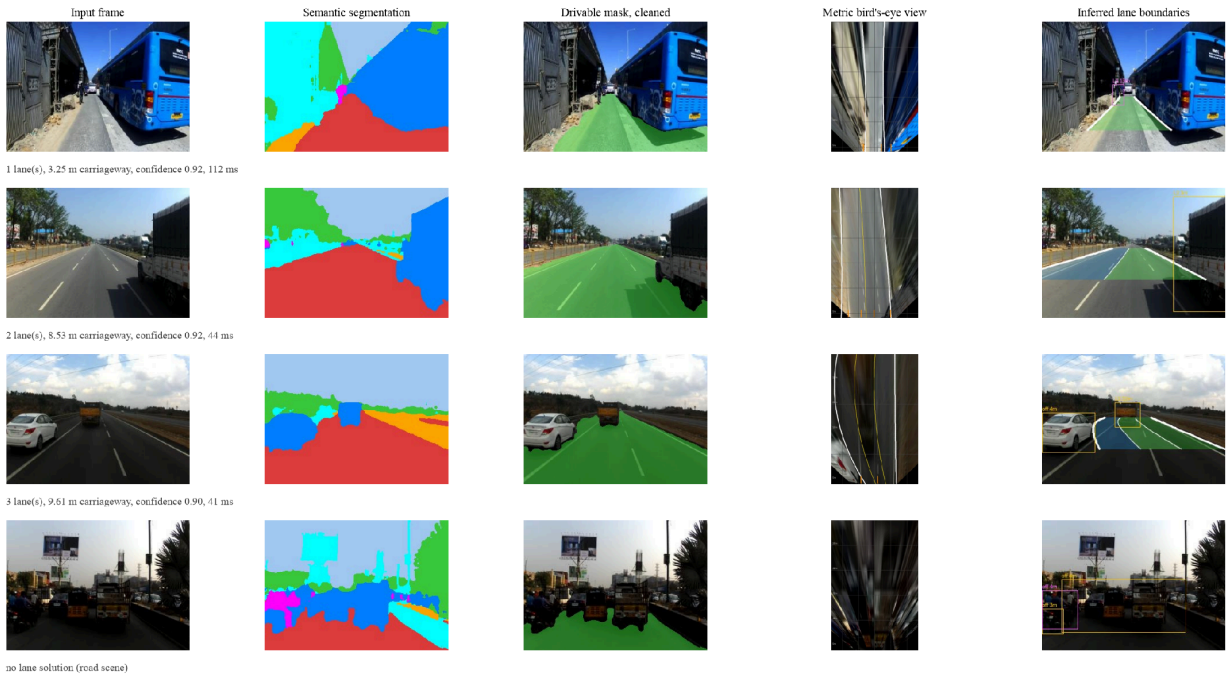

Figure 9. Pipeline stages on four IDD validation frames.

The bird's eye column is small in print because the rectified plane is rendered at the resolution the geometry actually uses. IDD-Lite frames are 227 by 320 pixels, so the far field of the rectified view is driven by very few source pixels, which is also why the overlay reach is capped at 22 metres.

Figures 18 and 19 in Section 4.3 show the same output inside the interface that serves it.

The scene check and the endpoint that serves a single frame:

```
@app.post("/api/infer")
async def infer(file: UploadFile = File(...),
                cameraHeight: float = Form(1.75),
                maxRange: float = Form(22.0)):
    img = cv2.imdecode(np.frombuffer(await file.read(), np.uint8),
                       cv2.IMREAD_COLOR)
    if img is None:
        raise HTTPException(400, "could not decode the image")
    p = pipeline(cameraHeight)
    r = p.process(img)
    return _finite(serialise(r, maxRange))
```

The `_finite` wrapper replaces non-finite floats with null before the response is encoded. Some metrics are genuinely undefined: boundary precision for the baseline that calls every pixel road has no predicted boundary to score, so it is NaN. The web framework encodes responses with NaN disallowed, so a single such value turned the whole results endpoint into a server error and the report page rendered blank with nothing visible to explain it. There is now a test that fails if that recurs.

CHAPTER 3: TESTING, VALIDATION AND RESULTS

3.1 Test plan

Testing runs at three levels. Unit tests cover the parts with a known correct answer, chiefly the projective geometry and the evaluation metrics, using synthetic inputs whose result can be derived by hand. Integration tests check that the API returns what the interface expects. System evaluation measures the whole pipeline on held out data and compares it against baselines.

The most important decision in the plan was to build the baselines first. Two of them do not look at the image at all: one calls every pixel road, and the other draws a fixed trapezoid in the lower half of the frame and returns it unchanged for every input. Nothing in a road perception pipeline should score below a fixed trapezoid, so the trapezoid is a floor. Without it there is no way to tell whether an output that looks plausible is any good, and Section 3.3.1 shows that this distinction mattered.

Tools: pytest for unit and integration tests, and a purpose built evaluation harness under `eval/` for the system measurements. The harness writes both Markdown and JSON to `outputs/results/`, and the application reads that JSON at run time, so a figure on the results page cannot drift away from the measurement that produced it.

3.2 Test cases

The suite contains 55 tests, all passing. Seventeen of them were added during the final audit, each pinning an input that had been found to return a server error or to draw something the measurements did not support. The table lists the cases that check behaviour a reader is most likely to doubt.

Table 6. Representative test cases, 20 of the 55 in the suite. All pass.

ID	What it checks	Input	Expected result	Status
T-01	Ground plane round trip	Point in the rectified plane	Returns to the same image point within tolerance	Pass
T-02	Vanishing point is the image of infinity	Synthetic parallel road edges	Recovered point matches the analytic one	Pass
T-03	Lateral scale independent of focal length	One scene, focal length swept	Recovered width varies under tolerance	Pass

T-04	Lateral scale proportional to camera height	Camera height doubled	Recovered width doubles	Pass
T-05	Constant width road stays constant	Synthetic straight road	Width variance across rows near zero	Pass
T-06	Border clipped edges are excluded	Road running off the frame	Clipped samples dropped from the fit	Pass
T-07	Vanishing point rejected when road touches both borders	Full width road	No vanishing point returned	Pass
T-08	Pitch is bounded	Degenerate edge geometry	Pitch clamped to plus or minus 15 degrees	Pass
T-09	Robust line fit ignores outliers	Edge points with 25 per cent noise	Fit matches the clean line	Pass
T-10	IoU on a hand computed case	Hand computed masks	Metric equals the hand computed value	Pass
T-11	Absent class is undefined rather than zero	Class missing from both	Returns NaN, not 0.0	Pass
T-12	Boundary F1 on perfect and disjoint masks	Identical, then disjoint	Returns 1.0, then 0.0	Pass
T-13	Trajectory recovers a known lane count	Synthetic 2, 3 and 4 lane traffic	Correct count recovered	Pass
T-14	Trajectory centres are accurate	Synthetic sequences	Centres within 0.5 m of truth	Pass
T-15	Trajectory survives off lane clutter	25 per cent stray positions	Count still recovered	Pass
T-16	Trajectory abstains without evidence	Sparse traffic	Returns none rather than guessing	Pass
T-17	Geometry preferred when trajectory is unconfident	Conflicting inputs	Geometric estimate retained	Pass
T-18	Implausible spacing rejected	Spacing far from the design range	Empirical estimate discarded	Pass
T-19	Results payload is JSON encodable	Live results directory	Encodes with NaN disallowed	Pass
T-20	Undefined metrics become null	Payload containing NaN	NaN replaced by null, values kept	Pass

3.3 Results and analysis

3.3.1 Drivable area, against trivial baselines

Table 5 and Figure 10 give the full ablation on the 204 frame IDD-Lite validation split. Every configuration was run on the same frames with the same harness.

Table 7. Drivable area evaluation on the IDD-Lite validation split.

Configuration	IoU	Precision	Recall	Boundary F1	FPS
Trivial: every pixel is road	0.3179	0.3179	1.0000	-	not meaningful
Trivial: fixed trapezoid, ignores the image	0.6450	0.8246	0.7476	0.1957	not meaningful
Study project: HSV threshold only	0.4131	0.4682	0.7783	0.2093	2,336
Study project: YOLOP as it was wired	0.1667	0.9119	0.1694	0.1382	6.4
Study project: full pipeline	0.4270	0.4747	0.8093	0.2048	6.3
YOLOP, same weights, wired correctly	0.7243	0.9923	0.7284	0.2205	6.2
YOLOP with CV blending	0.6979	0.9181	0.7443	0.2324	5.8
Capstone: classical only	0.5697	0.7709	0.6858	0.2099	361.0
Capstone: IDD-trained segmentation	0.9403	0.9821	0.9568	0.8343	67.5
Capstone: with mask cleanup	0.9310	0.9855	0.9440	0.7754	58.0
Capstone: with CV blending	0.9066	0.9719	0.9310	0.7234	50.7
Capstone: with blending and cleanup	0.9047	0.9781	0.9234	0.7031	50.3

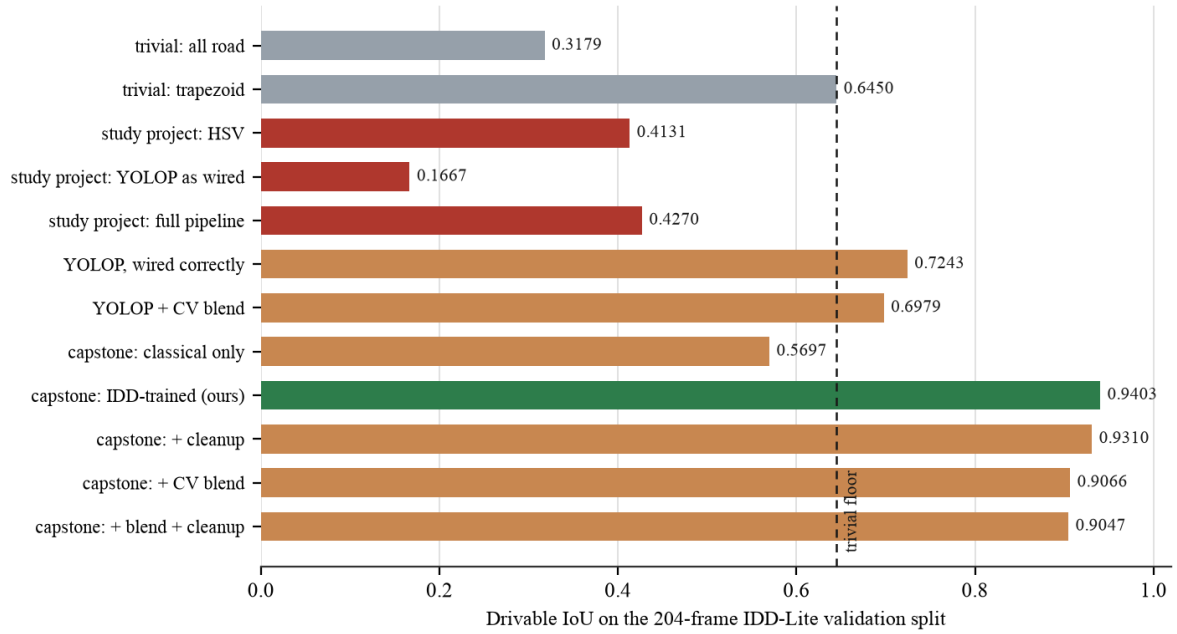

Figure 10. Drivable area ablation across twelve configurations.

Boundary F1 measures agreement on the outline rather than the area. A predicted boundary pixel counts as matched if a ground truth boundary pixel lies within three pixels of it, measured at the 227 by 320 evaluation resolution, and the score is the harmonic mean of precision and recall under that tolerance. It is reported alongside IoU because a mask can score well on area while its edges wander, and the edges are what the geometry stage consumes. The measure follows the boundary based evaluation argued for by Csurka and colleagues [6].

One row needs identifying before the rest can be read. "Capstone: IDD-trained segmentation" at 0.9403 is the segmentation head on its own, which is the ceiling the rest of the pipeline works under. The deployed system is the row below it: mask cleanup runs by default, so what the application actually produces is 0.9310 drivable IoU and 0.7754 boundary F1. Cleanup costs 0.0093 IoU and it is kept anyway, because the geometry stage needs a connected, hole-free mask rather than the most accurate set of pixels: with cleanup the pipeline returns a lane model on 165 of 204 frames, without it on 162. That is the trade, and it is a deliberate one.

Four further results in that table are worth stating plainly.

The study project pipeline scored 0.4270 against the trapezoid's 0.6450. A fixed shape that ignores the input entirely was more accurate. That comparison is what showed the problem

was structural rather than something parameter tuning would reach, and it is the reason the road detector was rebuilt rather than adjusted.

YOLOP was never broken; it was being given the wrong input. The study project fed it a stretched resize with no ImageNet normalisation. Letterboxed and normalised, the same weights move from 0.1667 to 0.7243 IoU, a factor of 4.3 recovered from preprocessing alone with no retraining. This is the most encouraging line in the table, because the component was sound the whole time.

The HSV threshold had high recall and low precision, 0.7783 against 0.4682. It found the road and also found a great deal that was not road. That combination is exactly what looks convincing in an overlay while scoring badly, which is how it survived visual inspection for two phases.

Every post-processing stage costs accuracy. Mask cleanup, confidence weighted blending with the classical branch, and the two together all score below the learned segmentation on its own. Section 6.3 returns to what that means for the hybrid design the project started with.

3.3.2 Effect of the training set

The shipped model is trained on IDD Segmentation 20k Part I: 6,993 training and 981 validation images downscaled to 288 by 512, written throughout as height by width. The schedule ran to completion over 40 epochs in 127 minutes on the laptop GPU, with the best result at epoch 35.

Table 8. Training configuration.

Training configuration	Value
Optimiser	AdamW, cosine decay with linear warm-up
Learning rate	4e-4
Batch size	16
Input size	288 by 512 (height by width)
Loss	Weighted cross entropy plus Dice, auxiliary deep supervision
Class weighting	Inverse square root frequency
Lane term	Binary cross entropy with positive weighting, weight 0.5
Epochs	40, best at 35
Hardware	Apple MPS, 127 minutes total

Table 9. Effect of training set size, measured on the IDD-20k validation split.

Metric	IDD-Lite (1,403)	IDD-20k (6,993)	Change
Semantic mIoU	0.6833	0.7574	plus 10.8 per cent

Drivable IoU	0.9326	0.9538	plus 2.3 per cent
Lane marking IoU, agreement with the teacher	0.5480	0.6311	plus 15.2 per cent
Pixel accuracy	0.8788	0.9148	plus 4.1 per cent

These figures are on the 981 frame IDD-20k validation split, which is a different and larger split from the 204 frame IDD-Lite one used for the comparative evaluation. The same model measures 0.7133 mIoU on IDD-Lite. The two numbers describe different data, not a discrepancy.

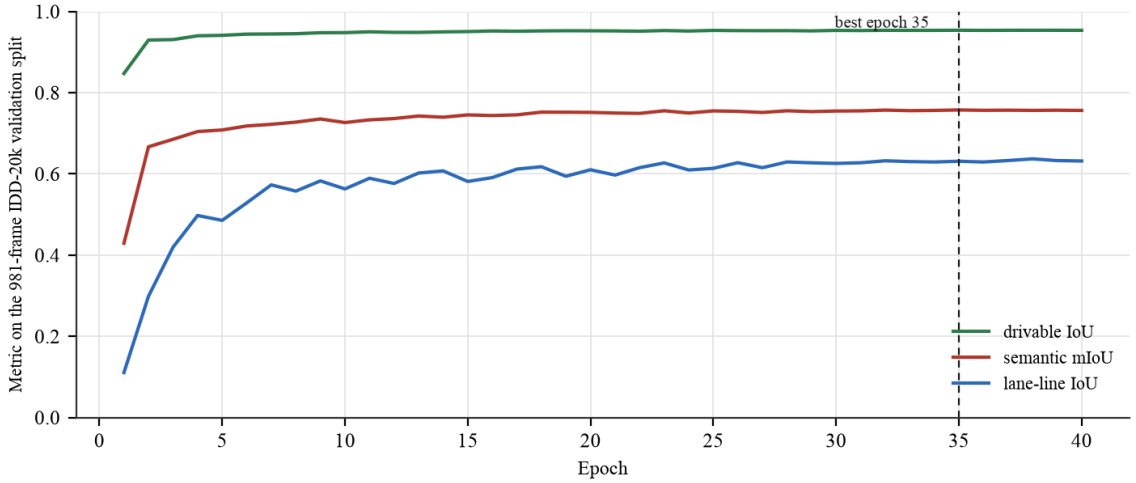

Figure 11. Validation metrics per epoch during training on IDD-20k.

The gains landed where the earlier analysis predicted they would. The two weakest classes were the two rarest, and the lane marking head had been described as resolution limited because a 150 millimetre marking is sub-pixel at 227 pixels of image height. Both improved most, while drivable IoU, already at 0.93 and close to its ceiling, moved least. The prediction was made before the run rather than after it.

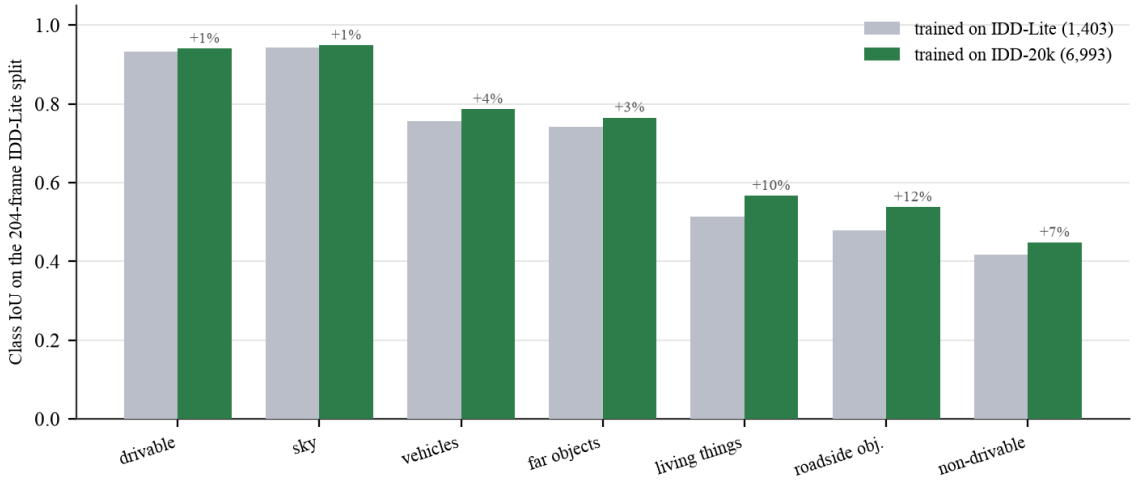

Figure 12. Per class IoU before and after scaling the training set.

One run was lost along the way, and the cause is worth recording. The first attempt at this schedule ran on a free cloud GPU and was destroyed at epoch 26 of 40 when the quota expired. Checkpoints were being written to the session's local disk and copied to durable storage only after training finished. A resume flag had been added specifically to make a dropped session cheap, and it could not help, because the checkpoint it resumes from sat in the one location a dropped session destroys. Adding resume while leaving the checkpoint on ephemeral storage is not a partial safeguard. The run was repeated locally and completed.

3.3.3 Is the inferred geometry correct?

This is the hardest question in the project and the one most easily avoided. Every figure quoted so far is either segmentation accuracy, which is a different task, or temporal stability, which says nothing about being right. This section measures correctness.

No Indian dataset carries lane annotations. But whatever the lane count is, the outermost inferred boundaries must coincide with the real edges of the road, and the drivable class ground truth gives those edges. Both are rectified into the same metric plane and compared at one metre intervals from 6 to 22 metres ahead. Samples where either reference edge is clipped by the frame are excluded, because there the reference is not a road edge either.

Table 10. Accuracy of the inferred carriageway against ground truth road edges.

Quantity	Median	Mean	p90
Left edge error	0.314 m	0.639 m	1.526 m
Right edge error	0.334 m	0.689 m	1.732 m
Centreline error	0.292 m	0.544 m	1.163 m
Carriageway width error	0.630 m	0.886 m	2.004 m
Width error, relative	11.8 per cent	22.4 per cent	46.5 per cent

These are per sample statistics over 2,288 samples. Each of the 159 comparable frames contributes at most 34 samples, two edges at each of the seventeen one metre steps. A sample is dropped where either reference edge is clipped by the frame or the canvas, because there the ground truth is not a road edge either, which is why 2,288 of a possible 5,406 survive. Four of the 165 solved frames contribute nothing at all, because fewer than four of their seventeen range steps had an unclipped reference edge on both sides, which leaves too little to fit against.

The median case is good and the tail is bad, and both statements are true. Half of all boundary samples fall within 31 centimetres of the true road edge at distances up to 22 metres. The mean sits far above the median and p90 approaches 4 metres, so the error distribution has a heavy tail: most frames are accurate and a minority are badly wrong.

The relative width error deserves its own comment, because a p90 of 46 per cent is the most alarming figure in this report. On the worst tenth of samples the recovered carriageway is wrong by more than one and a half times its true width. Those samples are not spread evenly: they concentrate in the same frames the confidence score is built to identify, and the gate below removes them along with the rest of the tail.

A heavy tail is tolerable if the system can identify it and unacceptable if it cannot. As originally written it could not. The confidence score correlated minus 0.187 with measured error in the analysis that motivated the rebuild, which is close to no relationship at all. Correlating candidate signals against measured error showed why.

Table 11. Signal correlation with measured boundary error.

Signal	Correlation with error	Used in the old score?
Edge polynomial fit residual	plus 0.435	No
Road users detected	plus 0.270	No
Vanishing point confidence	minus 0.251	No
Fraction of samples with both edges valid	minus 0.248	No
Width coefficient of variation	plus 0.188	Yes
Per edge coverage	minus 0.054	Yes, at 40 per cent weight
The old combined score	minus 0.187	n/a

The strongest available predictor was unused, and the largest single component of the score, at 40 per cent weight, carried almost no signal. The score was rebuilt from the four predictive signals, weighted by strength: 0.40 on the fit residual term, 0.25 on vanishing point

confidence, 0.20 on the both edges valid fraction and 0.15 on width consistency. The weights were chosen by predictor strength rather than fitted to the data.

Table 12. Confidence gating against error.

Confidence gate	Frames kept	Median error	p90 error
None	100 per cent	0.425 m	0.950 m
At least 0.6	91 per cent	0.413 m	0.884 m
At least 0.7	74 per cent	0.398 m	0.789 m
At least 0.8	37 per cent	0.339 m	0.594 m

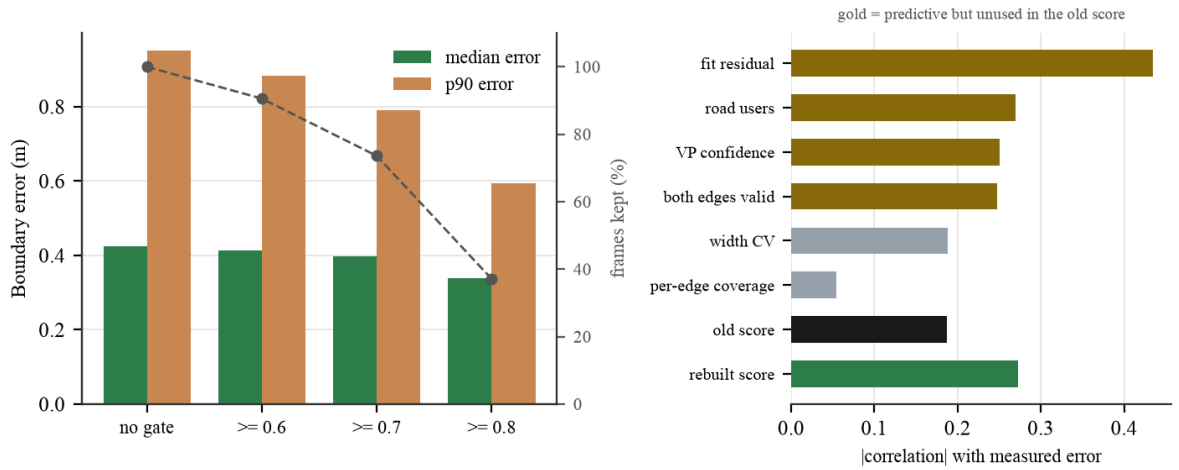

Figure 13. Confidence gating against measured boundary error.

Measured over all 159 scored frames the rebuilt score correlates 0.272 with boundary error, negatively, against 0.187 for the score it replaced. Gating at 0.7 keeps 74 per cent of frames and removes 6 per cent of the median error, so the tail is identifiable in advance rather than only in hindsight.

That correlation is weaker than the 0.470 an earlier draft of this report quoted, and the obvious reading of that change is the wrong one. The score did not get worse. The occlusion fix in Section 2.4.4 removed the large errors the score used to separate, so there is less variance left for any score to explain. Refitting the four weights against the corrected errors and validating on a held-out half of the split was tried, and it did not beat the shipped weights, 0.451 against 0.513, so the weights are unchanged. A weaker correlation over a much tighter error distribution is the better of the two outcomes, and reporting only the correlation would have hidden that.

One caution about mixing the two tables. Table 8 is per sample and Table 10 is per frame, where each frame contributes the median of its own samples. The per frame median is therefore the larger of the two, and the figures should not be quoted against each other.

What this does not validate is worth stating. It validates the carriageway extent, and through it the whole geometry stage. It does not validate the interior lane divisions on unmarked roads, for which no ground truth exists anywhere. That limitation is inherent to the problem rather than to this method.

3.3.4 Temporal stability

Measured on 150 sequences and 4,464 frames of IDD Temporal validation video at 960 by 540, unseen during training. Jitter is the median absolute change between consecutive frames. Over a three second clip the road ahead barely changes, so large swings are estimator noise rather than signal.

Table 13. Temporal stability over 150 IDD Temporal sequences.

Measure	Per frame	With Kalman and vote	Change
Carriageway width jitter, p50	0.486 m	0.054 m	minus 89 per cent
Carriageway width jitter, p90	4.066 m	0.342 m	minus 92 per cent
Centreline offset jitter	0.876 m	0.176 m	minus 80 per cent
Heading jitter	0.1248	0.0226	minus 82 per cent
Lane count flips per 100 frames	28.29	5.86	minus 79 per cent
Solution rate	85.1 per cent	88.9 per cent	plus 3.8 points

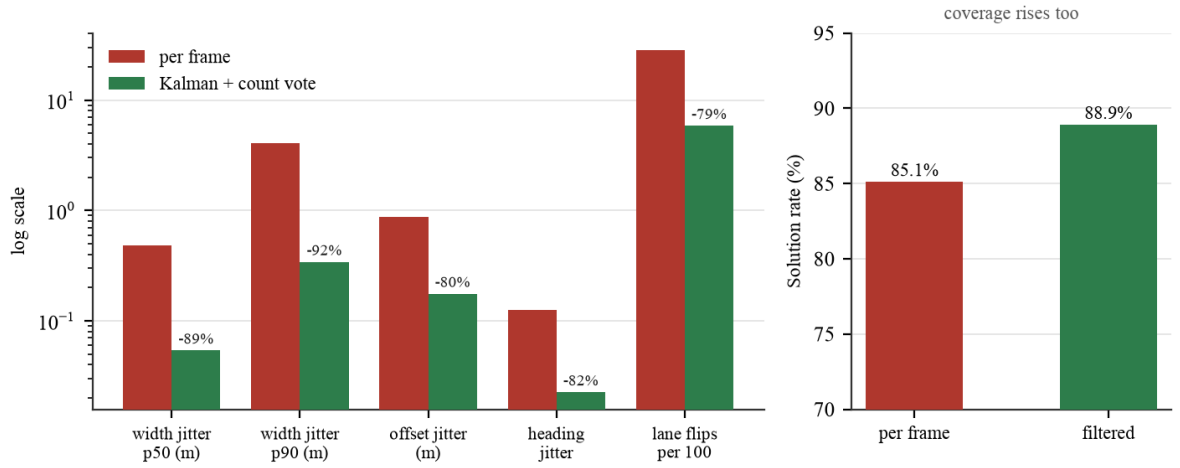

Figure 14. Effect of temporal filtering on stability and coverage.

The last row is the control that makes the rest meaningful. A filter that had stopped tracking its input would score perfectly on jitter while the solution rate stayed flat or fell. Instead, holding the previous estimate through a bad frame recovers 3.8 percentage points of coverage, so the filter is smoothing rather than ignoring. The p90 figure is the one to read:

typical frames were already fairly stable, and what the filter removes is the tail, where a single bad segmentation swung the estimated carriageway by four metres between consecutive frames.

3.3.5 Sensitivity and the rectification mask

Camera height is the only metric free parameter and lateral scale is exactly proportional to it, so its effect can be swept.

Table 14. Sensitivity of lane inference to the assumed camera height.

Camera height	Frames solved	Median carriageway	Median lane width	Mean lane count
1.30 m	146 of 204	4.67 m	3.41 m	1.58
1.50 m	156 of 204	4.97 m	3.49 m	1.60
1.75 m	165 of 204	5.27 m	3.41 m	1.68
2.00 m	171 of 204	5.90 m	3.51 m	1.86
2.40 m	166 of 204	6.98 m	3.54 m	2.04

Coverage rises with the assumed height, and that is a property of the estimator rather than evidence about the height. A frame is refused when the recovered carriageway falls below the 2.5 metre floor, and a larger assumed height scales every width up, so fewer frames are refused. A high coverage number at a large assumed height is therefore not a sign that the height is right.

Median lane width is the figure to watch, and it barely moves: 3.41 to 3.54 metres across the whole sweep against a design nominal of 3.5. That happens because the lane count is an integer division: as the carriageway grows the inferred count grows with it and the quotient stays near the nominal. It is a genuine robustness property of partitioning rather than thresholding, and it means the structure the system reports degrades gracefully under scale error even though the absolute widths do not.

What does move is the lane count, from 1.58 to 2.04 on average. Lane counts should therefore be read as conditional on the stated camera height, and a deployment that needs them exactly should fix the scale against one known width.

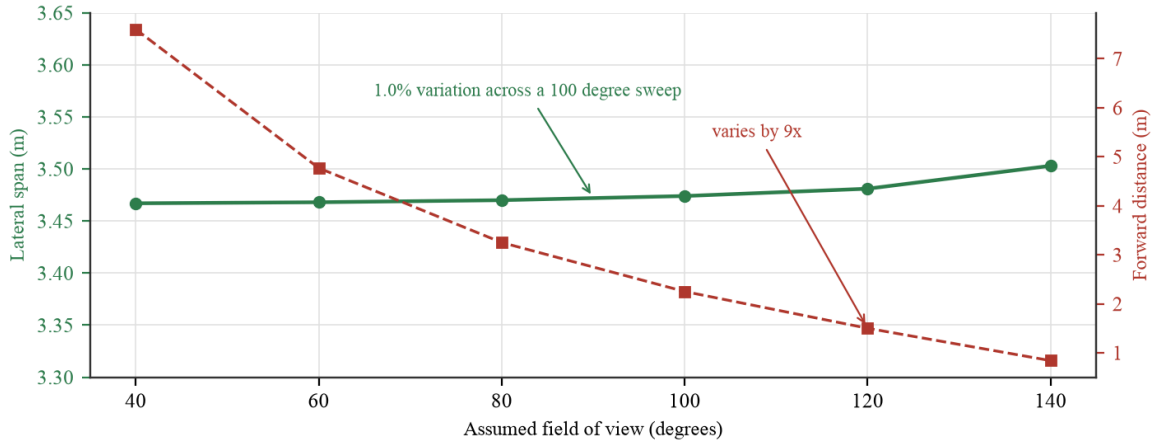

Figure 15. Lateral scale is independent of the assumed field of view.

Five rectification mask variants were measured on ground truth masks, so the geometry stage is evaluated independently of model error. Width CV is the standard deviation over the mean of the recovered width sampled every metre from 6 to 22 metres; a correct rectification of a straight road gives a near constant width, so lower is better, and the statistic is a ratio and therefore free of the metric calibration.

Table 15. Rectification mask variants.

Mask used for vanishing point	VP recovered	Median width CV	Dense traffic
A. Drivable, raw	184 of 204	0.2270	0.3094
B. Drivable with holes filled (adopted)	186 of 204	0.2306	0.3039
C. Plus bridged vehicle occlusions	185 of 204	0.2414	0.3721
D. Plus bridged vehicles and people	185 of 204	0.2526	0.4447
E. Road corridor, vehicles unioned in	152 of 204	0.3748	not measured

Variant B is adopted, and the choice is a trade rather than a clean win. Its median width consistency is marginally worse than variant A, 0.2306 against 0.2270, and it is better on the two things that matter more: it recovers a vanishing point on two more frames, 186 against 184, and it is the best of the five in dense traffic at 0.3039. Filling enclosed holes costs almost nothing and removes a failure mode where a shadow inside the carriageway splits the mask. Variant E was not measured in dense traffic because it already fails to recover a vanishing point on a quarter of all frames, which settles it without a further breakdown.

Dense traffic cannot be solved from a single frame. When the carriageway is not visible its edges cannot be estimated, and no masking strategy recovers information that is not present. The correct treatment is temporal, which is what Section 3.3.4 measures.

3.3.6 Failure analysis

Lane inference produces no solution on 39 of the 204 validation frames. Attributing each one is more useful than the headline coverage figure, because two of the three causes are the system behaving correctly.

Table 16. Attribution of frames with no lane solution.

Cause	Frames
Carriageway measured below the 2.5 m floor	20
Road edges not visible	11
No vanishing point recovered	8
Total refused	33

The largest single cause is a carriageway that measures below the minimum plausible lane width. That happens where the visible road is a narrow strip, either because traffic hides most of it or because only a sliver of surface falls inside the frame. Refusing there is correct: a lane model built on a 2.5 metre carriageway would be arithmetic performed on noise.

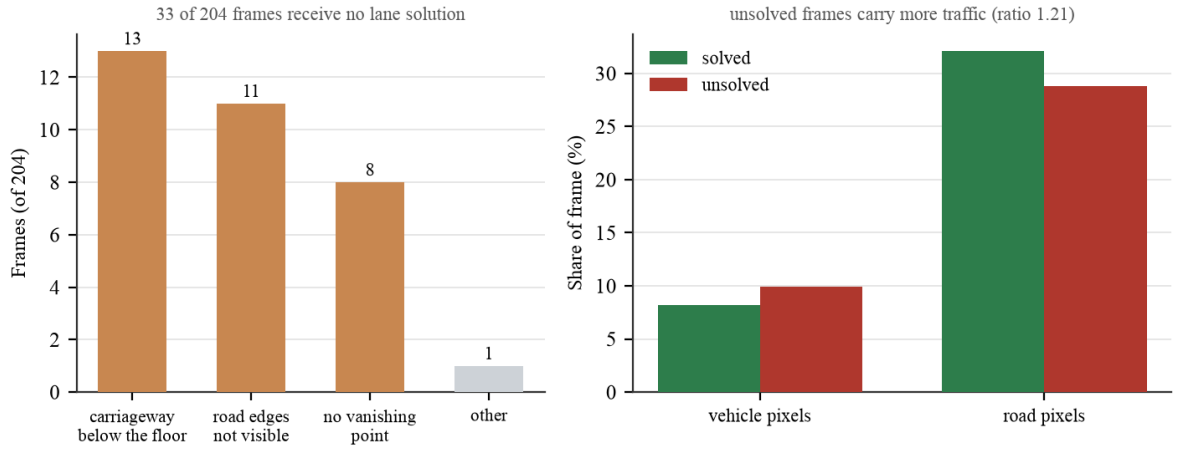

Figure 16. Attribution of frames that receive no lane solution.

Traffic density separates the frames that receive an answer from the frames that do not. Unsolved frames average 10.2 per cent vehicle pixels against 8.2 per cent for solved ones. It does not separate accurate answers from inaccurate ones: among solved frames the best fifth by error averages 10.7 per cent vehicle pixels against the worst fifth's 9.2, which is the wrong way round to be an explanation.

This reverses what we reported one draft ago, and the reversal has a cause we can point at rather than a correlation we can only observe. Dense traffic used to corrupt the width measurement through the bridging step described in Section 2.4.4, so busy frames returned confident wrong answers and dominated the error tail. Those frames are now refused instead of answered. Traffic still makes a frame hard; it has stopped making the answer wrong.

Table 17. Frames with a solution, split into fifths by boundary error.

	Best fifth	Worst fifth
Boundary error	0.181 m	1.199 m
Reported confidence	0.79	0.68
Vehicle pixels	10.7 per cent	9.2 per cent
Road pixels	31.1 per cent	32.8 per cent

The road pixel row runs the other way from what one might expect: the worst fifth shows more visible road, 32.8 per cent against 31.1. That is consistent with the largest refusal cause and with Section 3.3.3. A carriageway filling more of the frame is more likely to run off its sides, and where the observed boundary is the image border rather than the road, the edge fit degrades even though there is plenty of road in view. More road visible is not the same as more road boundary visible, and it is the boundary the geometry needs.

Two things follow. Traffic decides whether a frame can be answered at all, and it no longer decides whether the answer is any good, because the frames where it used to do damage are now refused rather than answered. The honest summary of three drafts is that we have twice stated a confident conclusion about traffic from a correlation and twice had to withdraw it; what changed this time was a mechanism we could name and fix, not a better fit. Second, what does still predict error is the edge-fit residual the confidence score already carries, 0.79 on the best fifth against 0.68 on the worst, and Section 3.3.3 shows that gating on it removes most of what remains of the tail.

CHAPTER 4: EXECUTION AND DEPLOYMENT

4.1 Execution environment

Table 18. Execution environment.

Component	Requirement
Python	3.12
Disk	About 2 GB for dependencies, 14 MB for the shipped weights
Accelerator	Optional. Apple MPS or CUDA if present, otherwise CPU
Segmentation, GPU	15.8 ms per forward pass on Apple MPS
Segmentation, CPU	75 ms per forward pass on two threads
Full pipeline, GPU	36.5 ms per frame, 27 frames per second
Browser	Any current browser. The camera tab also needs HTTPS

4.2 Deployment

Local installation and launch:

```
python3 -m venv .venv
./.venv/bin/pip install -r lane_inference/requirements.txt
./run.sh
```

The launcher builds the frontend on first run and serves the application at localhost. `./run.sh --lan` binds all interfaces so a phone on the same network can reach it, with one caveat: browsers permit camera access only over HTTPS or on localhost, so the live camera tab will not function over a plain address on the local network.

For hosted deployment the repository carries a container recipe under `deploy/`. The image installs PyTorch from the CPU wheel index, since the default wheel bundles CUDA and is roughly four times larger for no benefit on a CPU host, and it excludes the teacher model because no endpoint uses it at run time. The bundle that gets deployed is 29 MB.

One constraint follows from the design rather than the tooling. The application keeps per client filter state in process memory between requests, and writes rendered video to disk for a later request to serve. It therefore needs a host that runs a persistent process, and cannot be deployed as stateless serverless functions without moving both to an external store.

The API surface referred to throughout this report is the following.

Table 19. HTTP endpoints.

Endpoint	Method	Purpose
/api/health	GET	Device, loaded checkpoint, input size, class list.
/api/infer	POST	One image in, lane model and five rendered views out.
/api/live	POST	One camera frame, using per session temporal state.
/api/live/reset	POST	Clear the filter state for a session.
/api/stages	POST	Every intermediate image from one frame.
/api/compare	POST	Study project and capstone pipelines on one frame.
/api/video	POST	Process a clip, return an H.264 result and a series.
/api/media/{name}	GET	Serve a rendered video.
/api/samples	GET	List bundled sample frames or clips.
/api/sample/{kind}/{stem}	GET	Serve one sample.
/api/results	GET	Every measured result, read from outputs/results/.

Memory is the constraint that scales with use rather than CPU. The model is loaded once and shared; what grows per client is the session entry holding the Kalman state and the previous geometry, which is a few kilobytes. The video endpoint is the exception, since it decodes a whole clip and writes a rendered file, so concurrent video requests are the case to watch on a small host.

4.3 Demonstration

The interface has two routes, reached from a landing page. The report route presents the technical account with every headline figure read live from the evaluation output rather than typed into the page. The demonstration route has six tabs: analyse a single image, walk through each pipeline stage, use a live camera, process a video clip with temporal smoothing on or off, compare the study project pipeline against this one behind a drag to reveal slider, and read the measured results.

Figure 17. A consumer dashcam frame at dusk.

Figure 17 shows the system on a consumer dashcam frame at dusk, which is outside the training distribution. Section 6.3 reports what was measured when we tried to close that gap.

Figure 18 shows the landing page and the report route as they render in a browser. The report route is set on a light ground and the demonstration on a dark one, because camera frames and segmentation overlays are hard to read against white. The tab labelled "Phase 3 vs 4" in the interface is the comparison this report calls the study project against the capstone; phase numbering is the project's internal naming and is kept in the running application.

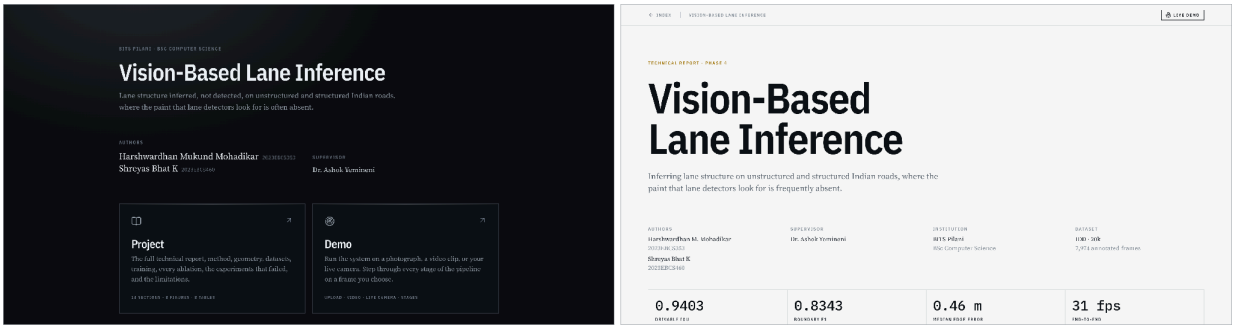

Figure 18. The landing page and the report route.

Figure 19 shows the Analyse tab after one validation frame has been processed. The readout gives lane count, carriageway width, lane width and latency, with the confidence ring beside a sentence that says what that confidence means for boundary error. The panel on the left states the two assumptions the metric output depends on, the camera height and the measured pitch, so a reader can see what the numbers rest on without leaving the page.

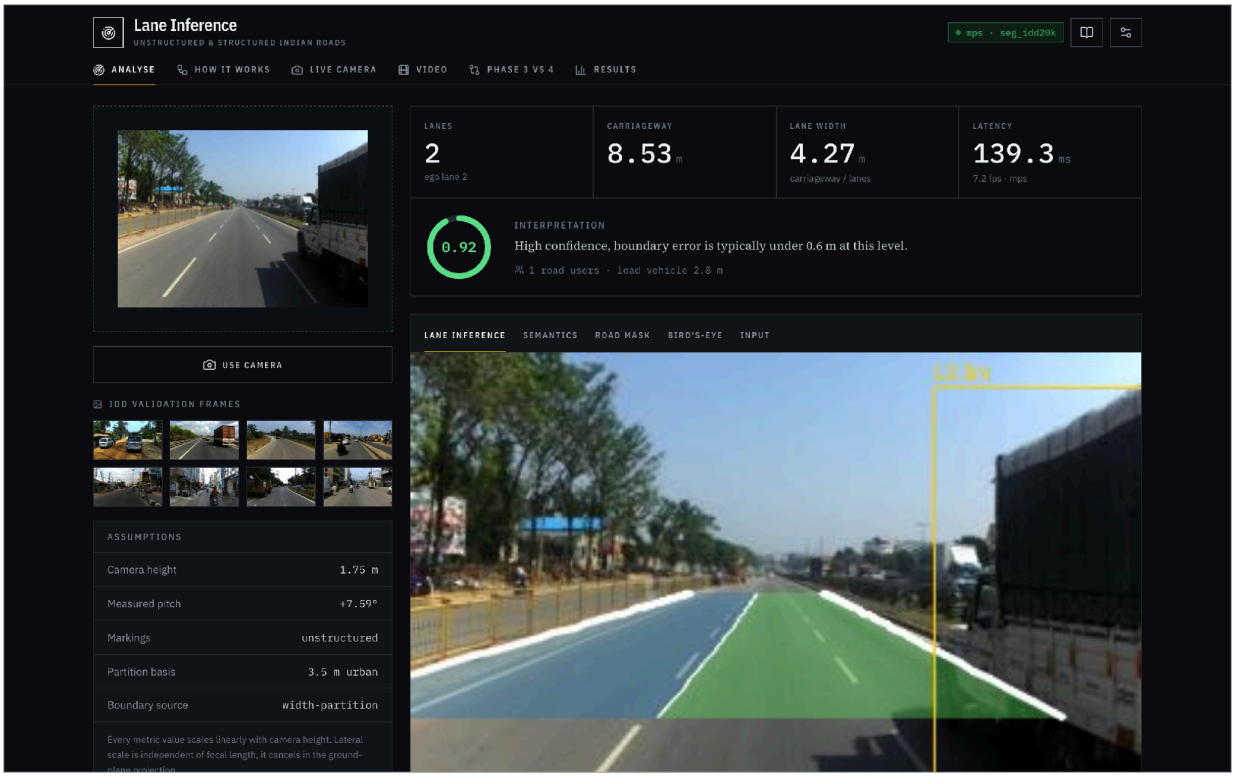

Figure 19. The Analyse tab after processing one validation frame.

Demonstration video: https://github.com/Arkanobot/VisionBasedLaneInference/blob/main/outputs/demo_video/capstone_demo.mp4

CHAPTER 5: PROJECT EXECUTION EVIDENCE

The three items in this chapter are records of process rather than results. They are filled in from the project repository and the supervisor review log, and the fields left blank below are the ones that must be completed from those records before submission rather than reconstructed from memory.

5.1 Version control evidence

The project was developed as a numbered progression of working versions, each one kept on disk when the next was started. That is the history, and it is a record of what was learned rather than of what was typed.

Each version was a different approach rather than a revision of the one before it. The work began with classical computer vision and ended with a segmentation network trained on Indian road imagery, and between those two points almost nothing carries over: a colour and texture detector, a thresholding pipeline, an off the shelf detector wired into that pipeline, and finally a model of our own share the problem statement and very little else. Successive versions were therefore separate projects that answered the same question in different ways, not commits against a common trunk, and that is why a single repository carrying incremental updates was not maintained while the approach was still changing. The repository below holds the delivered system.

Table 20. Version history as kept on disk. Dates run from 27 June 2026 to 24 August 2026.

Version	Approach	Why it was superseded
lane_detector_project	Classical computer vision: colour and texture segmentation of the road surface	Not separable from shadow, wet tarmac and unpaved shoulder by colour alone
lane_detector_v2	The same pipeline with HSV thresholding and asphalt detection heuristics	Each heuristic fixed one scene and broke another; no single threshold generalised
lane_detector_v3	YOLOP dropped in as the drivable-area detector	Scored below the fixed trapezoid baseline; the wiring was suspected
lane_detector_v4	YOLOP with letterboxing and ImageNet normalisation corrected	A large gain that confirmed the wiring, still short of what the geometry needs
lane_detector_v5_dl	First segmentation network trained on IDD	Trained on a partial dataset root; superseded once the full split was prepared
lane_detector_v6_dl_fullDataset	The shipped architecture trained on the full IDD-20k split	Current. Carried forward into this capstone

Each version was started because the one before it had been measured and found short, and each failure was a failure of approach rather than of tooling. The first two versions failed because the drivable surface on an Indian road is not separable by colour and texture, which is a limit of classical computer vision and not of the implementation. The next two failed because a detector trained on other roads does not transfer to these ones, which we did not know at the outset and learned by measuring it. What each failure produced was the specific piece of knowledge the next version needed: that thresholds do not generalise, that the wiring of a pretrained model has to be verified before its accuracy means anything, and finally that the only remaining option was to train on Indian data ourselves.

None of this is asserted from memory. The approach behind every version is carried into the ablation in Section 3.3.1, measured on the same split with the same metrics, so the drivable IoU of each rejected approach sits in one table beside the system that replaced it, and the ablation figure in that section plots the same comparison.

Repository: https://github.com/Arkanobot/VisionBasedLaneInference

The source tree, the trained weights and the evaluation harness are distributed together. The dataset is not, for the reason given in the data statement: IDD is released through a registration process with a licence presented at download time. A reader who registers and downloads IDD-Lite can verify their copy against the checksums recorded in `tools/idd_lite_manifest.json` and then reproduce every measurement in Chapter 3 with the commands in Appendix B.

5.2 Weekly progress summary

The first eight weeks were spent on four approaches that were each built, measured and set aside. They are recorded here as they happened rather than compressed out, because the measurements that rejected them are what justified training a model of our own, and all four remain in the ablation in Section 3.3.1.

Table 21. Weekly progress summary.

Week	Task planned	Task completed	Supervisor remark
1	Frame the problem; build a classical road-surface pipeline	Colour and texture segmentation of the drivable surface; first end-to-end run	
2	Measure the classical pipeline against the validation split	Accuracy well short of what lane geometry needs; approach set aside	

3	Extend the classical pipeline with HSV thresholding	HSV road detector built; asphalt detection heuristics added	
4	Measure the extended classical pipeline	Fails on shadow, wet tarmac and unpaved shoulder; approach set aside	
5	Integrate YOLOP as a drop-in drivable-area detector	YOLOP wired into the pipeline and producing masks	
6	Measure the YOLOP-based pipeline on IDD	Below the fixed-trapezoid baseline; wiring suspected, approach set aside	
7	Correct the YOLOP configuration and re-measure	Letterboxing and ImageNet normalisation fixed; large gain but still short	
8	Decide between tuning YOLOP further and training on IDD	Reviewed with supervisor; agreed to train a custom model on IDD	Pivot to a custom IDD-trained model agreed
9	Train a segmentation network on IDD	MobileNetV3 with LR-ASPP and an FPN decoder trained on IDD-20k	
10	Recover metric geometry from the segmented surface	Vanishing point, pitch, inverse perspective mapping and carriageway partition	
11	Stabilise the output across video	Kalman filter over the lane parameters and a majority vote on lane count	
12	Build the evaluation harness and the application	Trivial baselines, the twelve-way ablation, confidence calibration and the web application	

5.3 Supervisor interaction summary

One formal review took place. It is recorded in full below, and no further supervisor review was held after it; that is stated here rather than left as empty rows.

Table 22. Supervisor interaction summary.

Review date	Topic discussed	Key feedback received	Action taken
27 June 2026	Progress on deep-learning-based lane inference, and the accuracy of the	YOLOP alone was not reaching the accuracy Indian road conditions require; pivot to	YOLOP dropped as the detector. A segmentation network was trained on IDD and

	YOLOP-based approach	training a custom model on IDD rather than relying on an off-the-shelf detector	became the shipped system; YOLOP was retained only as a baseline and as the teacher for the distilled lane head

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary of implementation

The project infers lane structure on Indian roads from a single forward facing camera, without requiring lane markings. A segmentation network labels the drivable surface, the vanishing point of its two boundaries fixes the camera pitch, the ground plane is rectified to metric units, and the measured carriageway is divided by the IRC design lane width to give a lane count. A second output head finds paint where paint exists, at no extra inference cost, and a Kalman filter stabilises the result across video. The whole system is delivered as a running web application.

6.2 Achievements

- Drivable area IoU of 0.9403 and boundary F1 of 0.8343 on the 204 frame validation split, against 0.4270 and 0.2048 for the pipeline this replaces.
- A metric lane count derived from a design standard rather than a fixed subdivision, producing a solution on 165 of 204 validation frames with a median road edge error of 0.31 metres over 2,288 samples.
- A demonstration that the lateral scale of the rectified plane does not depend on focal length, which removes a parameter the dataset cannot supply and leaves the pipeline with a single metric assumption.
- An overlay that stops where the evidence stops, closing a case in which the rendered picture claimed more than the measurements supported.
- A per row treatment of vehicle occlusion in the width profile. It was found by comparing the lane model against the mask it was built from rather than against ground truth, which localised the defect to the geometry stage instead of the segmentation, and it took the relative carriageway error at the ninetieth percentile from 151 per cent to 47 and halved the mean road edge error.
- A confidence score that correlates minus 0.272 with measured error against minus 0.187 for the score it replaced, so unreliable frames can be identified in advance.
- Temporal filtering that reduces width jitter by 89 per cent while raising coverage from 85.1 to 88.9 per cent, over 4,464 real video frames drawn from three drives.

- A lane marking head obtained by distillation, which raised the number of frames classified as structured from 27 to 39, a 44 per cent increase, for 0.011 mIoU on the semantic task. That cost is not cleanly attributable, for the reason given under Table 4.
- An evaluation harness with trivial baselines, which is what made every result above interpretable.
- Measured self correction. Three claims made confidently in earlier drafts did not survive re-measurement and are reversed in print rather than removed without comment: that traffic density does not explain failure and then that it does (Section 3.3.6), that two independent anchors agreed on a field of view (Section 2.4.2), the drivable IoU quoted for the deployed system (Section 3.3.1), and the carriageway anchor that appeared to support the assumed camera height (Section 2.4.2). The last of those was withdrawn because the measurement behind it was produced by the occlusion bug named above. Each correction and the measurement that forced it are in the text.

6.3 What did not work

Four ideas were implemented, measured and rejected. They are reported because the measurements are part of the contribution: a negative result that has been measured is worth more than a positive one that has been assumed.

6.3.1 Fusing the classical and learned branches

The hybrid design the project carried from its earliest phase is not supported by the data. Replacing the original bitwise OR with a principled confidence weighted sum still reduces accuracy, from 0.9403 to 0.9066 with the in domain model and from 0.7243 to 0.6979 with out of domain YOLOP. The classical branch is much weaker than either learned branch at 0.5697 IoU, so any fixed weight blend drags the result toward it. The classical branch is retained as a gated fallback, used only if the learned branch collapses, which on this data is equivalent to the learned branch alone while still covering model failure.

6.3.2 Bridging occlusions before the vanishing point

Vehicles fragment the road mask, so putting them back before fitting the road edges seems natural. Two versions were measured and both were rejected, for different reasons. Table 13 carries the numbers.

The blunt version, unioning the vehicle mask into the road mask so the whole corridor becomes one region, is variant E. It drops vanishing point recovery from 184 of 204 frames to

152, which is 90.2 per cent to 74.5 per cent. Parked roadside vehicles extend the region sideways into the frame border and recreate exactly the clipping that Section 2.4.3 removes.

The careful version, bridging only gaps genuinely covered by occluder pixels, is variant C. It avoids the border problem and recovers a vanishing point on 185 frames, essentially unchanged from the adopted variant B at 186. It fails on the metric that matters instead: width consistency in dense traffic degrades from 0.3039 under variant B to 0.3721 under variant C, because the bridged boundary follows the outer, partly occluded road edge, which is noisier to fit than the smaller but cleaner unoccluded fragment.

The same operation applied to the width stage is essential rather than harmful, as Section 2.4.4 describes. Separating the two uses is what made that visible, and it is the clearest case in this project of a negative result producing a positive one.

6.3.3 Inferring lanes from observed traffic

On a road without paint the lanes might be wherever traffic actually flows, so accumulating observed vehicle positions and finding modes in their lateral distribution should recover lane structure empirically. Sweeping the evidence threshold makes the estimator fire on 75 per cent of clips rather than 3.4 per cent of frames, but agreement with the geometric estimate stays flat at 34 to 40 per cent, and the recovered spacing holds at a median of 4.8 metres against the IRC range of 3.0 to 3.5, without tightening as more evidence is required.

That pattern rules out insufficient evidence as the explanation. If the modes were lanes seen through noise, demanding more observations would pull the spacing toward the design width. It does not. The modes are stable and reproducible, and they are not lanes. Offered as interpretation rather than measurement, the likely reading is that over a three second window the two dominant clusters are a vehicle ahead and an oncoming vehicle, which on a two lane road sit about five metres apart.

The negative result is bounded rather than general, which matters. The estimator does recover 2, 3 and 4 lane layouts from synthetic sequences with centres accurate to better than 0.5 metres and survives 25 per cent off lane clutter, and those cases are covered by tests T-13 to T-16. What has been shown is narrower: three second windows of real Indian traffic do not exhibit lane shaped lateral modes. Testing it properly needs continuous footage of tens of seconds, and IDD Temporal supplies 31 frame windows drawn from only three distinct drives.

6.3.4 Closing the consumer dashcam gap at test time

A frame from a consumer dashcam at dusk, wide angle, heavily vignetted and showing the vehicle's own bonnet, produced a poor result. Two genuine defects surfaced first and both were bugs rather than domain gap. Bottom connectivity assumed the road reaches the last row of the frame, which is true on IDD and false on any dashcam that sees its own bonnet; 48,461 pixels of correctly segmented carriageway were discarded while a 972 pixel false positive on the bonnet was kept. Rewriting the rule as low in the frame and large improved IDD as a side effect, from 0.9067 to 0.9258 drivable IoU. Those two figures come from the dashcam investigation record rather than Table 5, because they were measured on the pipeline configuration in use at the time; the shipped configuration in Table 5 reaches 0.9403. The second defect was the unchecked width estimate described in Section 2.4.7.

What remained is the training distribution rather than a bug. IDD is daylight footage from a normal lens. Four approaches were measured and all four rejected. They appear as five rows in Table 18 because the dual hypothesis approach was tried with two different selection rules.

One caveat about that table before reading it. The IDD baselines in the third column differ between rows, from 0.9329 to 0.9437, and so do the two dashcam baselines, 13.0 and 13.6 per cent. The investigation ran over several days while the connectivity and width defects described above were being fixed, and each remedy was measured against the code as it stood that day. Each row is therefore internally valid as a before and after pair, and the rows should not be compared with one another. Re-running all five against a single baseline is listed in Section 6.5.

Table 23. Remedies for the dashcam domain gap, all rejected. The first four act at test time; the last is a training-time intervention.

Approach	Effect on the dashcam frame	Effect on IDD
CLAHE applied unconditionally	Drivable 13.0 to 29.7 per cent	IoU 0.9437 to 0.8790
Adaptive CLAHE, triggered by image statistics	No trigger exists	n/a
Dual hypothesis, selected by lane confidence	Selects the wrong one	IoU 0.9329 to 0.9062
Dual hypothesis, selected by classical agreement	Selects correctly	IoU 0.9402 to 0.9002
Fine tuning with synthetic dashcam augmentation	13.6 to 13.5 per cent	0.9329 to 0.9303

Contrast enhancement works and cannot be applied. CLAHE more than doubles the recovered drivable area on this frame but costs 6.5 points of IoU on IDD, so it is only correct

on frames that need it. No image statistic identifies those frames: lightness standard deviation, percentile spread, median and dark pixel fraction were measured across the validation split against this frame, and it sits inside the normal range on every one of them.

Table 24. Image statistics: the dashcam frame against IDD.

Statistic	IDD p05	IDD p50	IDD p95	Dashcam frame
Lightness standard deviation	49.5	66.4	86.9	53.7
Lightness spread, p95 minus p05	152.7	215.5	248.7	178.0
Lightness median	50.0	92.5	126.2	87.0
Dark pixel fraction (0 to 1)	0.1	0.2	0.4	0.2

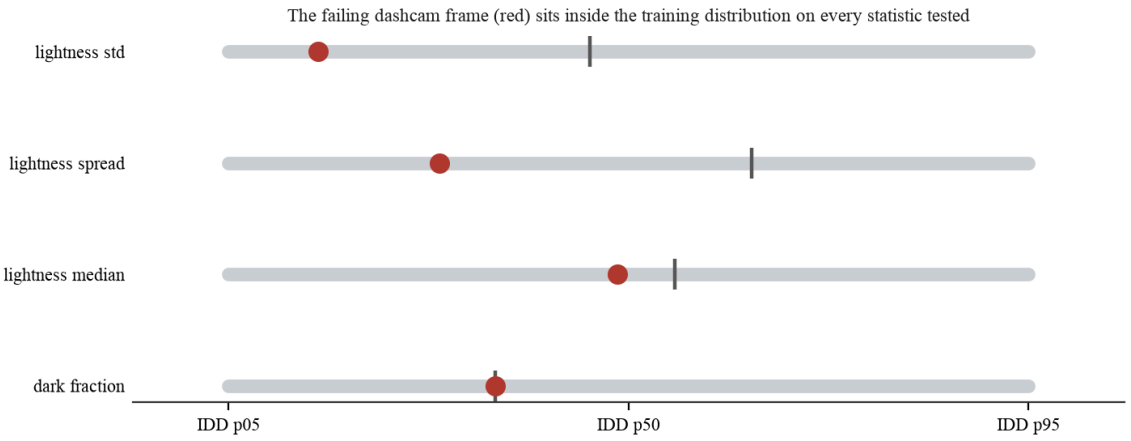

Figure 20. The failing dashcam frame against the training distribution.

The failure is therefore not low contrast. CLAHE helps for a different reason: it breaks up a large, smoothly varying hazy region that the network otherwise labels sky. A prediction side trigger was tested too, since sky predicted below the horizon is physically impossible, and it does separate, at 0.47 per cent on the dashcam against a maximum of 0.43 across the validation split, but by a margin far too narrow to gate on. Fine tuning on synthetic vignetting, non linear low light response and barrel distortion for twelve epochs moved the recovered drivable area from 13.6 to 13.5 per cent while costing 0.0026 IoU.

The honest statement of the limitation is that the system is trained on daylight, normal lens footage and degrades on wide angle sensors at dusk, and the fix is training data of that kind rather than a transform applied at test time.

6.4 Limitations

- **Metric scale rests on an assumed camera height of 1.75 metres.** Every metric output scales linearly with it. One known width at deployment removes the assumption.
- **Interior lane divisions are not validated on unmarked roads.** No ground truth exists. The carriageway extent is validated; its division rests on the IRC design width.
- **Half of the available annotated data is unused.** Part II of IDD-20k was downloaded and never merged, because the preparation script stopped at the first dataset root it found. The model therefore sees 6,993 of roughly 14,000 annotated frames. The defect is fixed and the retraining is outstanding.
- **Painted markings are found on a minority of frames, and the two counts quoted for this are not the same measurement.** The distillation ablation counted frames where the lane head fired at all, 27 before and 39 after, on the IDD-Lite split. The pipeline separately classifies a frame as structured only when the markings it finds are plausibly spaced, which holds on 24 of the 165 solved frames. The second is the stricter test and the one the partition uses. Both are low, and thin markings being resolution limited at the current input size is the likely reason.
- **Performance degrades on wide angle sensors in low light.** Section 6.3.4. The fix is training data, not a test time transform.
- **No test split evaluation, and the best epoch was chosen on the split it is reported on.** The IDD test split is unlabelled, so every figure here is a validation figure. Epoch 35 was selected because it maximised validation mIoU, and Table 5 then reports that same epoch on that same split, so 0.9403 should be read as a best-epoch-on-this-split number rather than a held-out one.
- **The narrow-road divisor of 3.0 metres is a project parameter, not a quoted clause.** The urban 3.5 metre width follows IRC:86. The narrow-road value was chosen against the plausibility floor rather than read from a standard, and it should be confirmed against the applicable rural or village road standard before any deployment claim rests on the lane counts it produces.
- **One training run, one seed, no confidence intervals.** Nothing here is repeated across seeds, so differences of a few thousandths between configurations in Table 5 should not be treated as significant.
- **The 150 temporal sequences are not 150 independent scenes.** IDD Temporal supplies 31 frame windows sampled around segmentation frames, drawn from only three distinct

drives. The 89 per cent jitter reduction in Section 3.3.4 is measured on 4,464 frames but it generalises only as far as those three drives do.

- **The model is trained at a higher resolution than the evaluation frames carry.** Training runs at 288 by 512 while IDD-Lite frames are natively 227 by 320, so every evaluation frame is upscaled before inference. That plausibly accounts for part of the lane marking head being the weakest output, since a 150 mm marking is sub-pixel at the source resolution and upscaling cannot recover it.
- **Ego lane assignment is reported but never evaluated.** The lane containing the camera is read off the partition by taking the interval that contains X = 0, so it is correct by construction only if the partition itself is. IDD carries no ego lane ground truth, so the output is unvalidated and is the first item in future work.
- **Road user distances are reported but never evaluated.** The interface shows a lead vehicle distance and nearest object distances, and Section 1.5 lists them in scope. No ground truth for them exists in IDD, so they are unvalidated output and should be read as indicative.
- **Wrong side detection is implemented but unvalidated and undocumented.** It requires motion, so it is video only, and IDD carries no ground truth for it. It is disabled by default and is not described in Chapter 2 because there is no measurement to describe.

6.5 Future enhancements

Table 25. Outstanding work, in order of value.

Work	Why it matters	Estimated effort
Merge Part II of IDD-20k and retrain	Roughly doubles the training set. Table 7 shows what more data bought last time.	4 hours, mostly unattended
Train on IDD-AW, the adverse weather subset	The direct remedy for Section 6.3.4 and the one substantive experiment left open.	5 hours
Retrain at 384 by 640	Structured road detection is resolution limited; this is the obvious lever.	3 hours
Validate ego lane assignment	Implemented but never checked, so it is currently an unverified claim.	2 hours
Controlled multi-task comparison	Retrain single and multi-task under identical hyperparameters so the 0.011 mIoU gap can be attributed rather than assumed.	3 hours
Re-run the five dashcam remedies against one baseline	Table 20 measured each against the code as it stood that day, so the rows cannot be compared	2 hours

	with one another. See Section 6.3.4.	
Confirm the narrow-road divisor against the applicable standard	The lane count for every carriageway under 6.0 m rests on it.	1 hour
Calibrate against one known width in the field	Removes the last metric assumption entirely.	1 day of field work

Of these, merging Part II is the highest value item, because the effect of training set size on this problem is already measured rather than assumed.

REFERENCES

[1] G. Varma, A. Subramanian, A. Namboodiri, M. Chandraker and C. V. Jawahar, "IDD: A dataset for exploring problems of autonomous navigation in unconstrained environments," in Proc. IEEE Winter Conf. on Applications of Computer Vision (WACV), 2019. [Online]. Available: https://idd.insaan.iiit.ac.in/

[2] D. Wu, M. Liao, W. Zhang and X. Wang, "YOLOP: You only look once for panoptic driving perception," Machine Intelligence Research, vol. 19, pp. 550 to 562, 2022. [Online]. Available: https://github.com/hustvl/YOLOP

[3] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le and H. Adam, "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV), 2019.

[4] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan and S. Belongie, "Feature pyramid networks for object detection," in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2017.

[5] H. A. Mallot, H. H. Bulthoff, J. J. Little and S. Bohrer, "Inverse perspective mapping simplifies optical flow computation and obstacle detection," Biological Cybernetics, vol. 64, no. 3, pp. 177 to 185, 1991.

[6] G. Csurka, D. Larlus and F. Perronnin, "What is a good evaluation measure for semantic segmentation?" in Proc. British Machine Vision Conf. (BMVC), 2013.

[7] Indian Roads Congress, IRC:86-1983, Geometric Design Standards for Urban Roads in Plains. New Delhi, India: Indian Roads Congress, 1983.

[8] Indian Roads Congress, IRC:73-1980, Geometric Design Standards for Rural (Non-Urban) Highways. New Delhi, India: Indian Roads Congress, 1980.

[9] R. E. Kalman, "A new approach to linear filtering and prediction problems," Journal of Basic Engineering, vol. 82, no. 1, pp. 35 to 45, 1960.

[10] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in Graphics Gems IV, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474 to 485.

[11] G. Hinton, O. Vinyals and J. Dean, "Distilling the knowledge in a neural network," in NIPS Deep Learning Workshop, 2015.

[12] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan and T. Darrell, "BDD100K: A diverse driving dataset for heterogeneous multitask learning," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2020.

[13] A. Paszke et al., "PyTorch: An imperative style, high performance deep learning library," in Advances in Neural Information Processing Systems 32, 2019.

[14] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[15] X. Pan, J. Shi, P. Luo, X. Wang and X. Tang, "Spatial as deep: Spatial CNN for traffic scene understanding," in Proc. AAAI Conf. on Artificial Intelligence, 2018.

[16] D. Neven, B. De Brabandere, S. Georgoulis, M. Proesmans and L. Van Gool, "Towards end-to-end lane detection: an instance segmentation approach," in Proc. IEEE Intelligent Vehicles Symposium (IV), 2018.

[17] Z. Qin, H. Wang and X. Li, "Ultra fast structure-aware deep lane detection," in Proc. European Conf. on Computer Vision (ECCV), 2020.

[18] L. Tabelini, R. Berriel, T. M. Paixao, C. Badue, A. F. De Souza and T. Oliveira-Santos, "PolyLaneNet: Lane estimation via deep polynomial regression," in Proc. Int. Conf. on Pattern Recognition (ICPR), 2020.

[19] T. Zheng, Y. Huang, Y. Liu, W. Tang, Z. Yang, D. Cai and X. He, "CLRNet: Cross layer refinement network for lane detection," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2022.

[20] India Driving Dataset (IDD), INSAAN, IIIT Hyderabad. [Online]. Available: https://idd.insaan.iiit.ac.in/ (registration required; the licence is presented at download). This is the source the data was obtained from; [1] is the paper that describes it.

APPENDIX

A. User manual

Open the application at the address the launcher prints. The report route presents the technical account. The demonstration route has six tabs.

- **Analyse.** Drop in a road photograph or pick one of the bundled validation frames. The panel reports lane count, carriageway width, lane width, ego lane and confidence, and the view selector steps through every intermediate stage.
- **How it works.** Walks the same frame through the pipeline one stage at a time, with a note explaining what each stage does.
- **Live camera.** Streams frames from the device camera. Requires HTTPS or localhost. Readouts are laid over the frame so they can be read while the camera is pointed at a road.
- **Video.** Processes an uploaded clip, with temporal smoothing switchable so its effect can be seen directly.
- **Study project versus capstone.** Runs both pipelines on one frame behind a drag to reveal slider.
- **Results.** Every measurement, read live from the evaluation output, with each written record readable in the page.

Two controls sit behind the settings icon. Camera height is the only metric assumption the pipeline makes and every distance scales linearly with it. Overlay reach caps how far ahead boundaries are drawn, because beyond about 22 metres the rectification is driven by very few pixels.

B. Installation guide

```
# 1. dataset: register at idd.insaan.iiit.ac.in, accept the licence,
#    download IDD-Lite, then verify the copy against our manifest
python tools/fetch_dataset.py --archive <downloaded archive>

# 2. environment
python3 -m venv .venv
./.venv/bin/pip install -r lane_inference/requirements.txt

# 3. run the application
./run.sh                 # localhost only
./run.sh --lan           # also reachable on the local network

# 4. reproduce the results in Chapter 3
export PYTHONPATH=lane_inference
./.venv/bin/python -m eval.evaluate        # Table 5, the ablation
```

```
    ./.venv/bin/python -m eval.lane_geometry   # Table 8, boundary error
     ./.venv/bin/python -m eval.temporal          # Table 11, needs the
clips
    ./.venv/bin/python -m eval.sensitivity     # Table 12
    ./.venv/bin/python -m eval.failure_analysis  # Table 14

    # 5. run the test suite
    ./.venv/bin/python -m pytest lane_inference/tests -q
```

Each evaluation command rewrites the corresponding file under `outputs/results/`, and the application reads those files at run time, so a reader who re-runs a measurement will see the interface update to match.

C. Source code

Repository: https://github.com/Arkanobot/VisionBasedLaneInference

The distributed archive contains the pipeline, the server, the interface source and its build, four trained checkpoints, every measured result file, the evaluation harness and the test suite. It does not contain the dataset. IDD is distributed by IIIT Hyderabad through a registration process with a licence presented at download time, so neither it nor any subset derived from it is passed on here. Run `python tools/fetch_dataset.py --archive <path>` after registering and downloading, and it will extract the data and verify it against the checksums every reported result was measured on.

D. Demonstration video

Video link: https://youtu.be/yfWKEdWGo0c